ANALYTICS CULTURE ZEEN

VOL. 2 | 2026

SQL 101: РЕЦЕПТЫ ПРОДУКТОВОГО АНАЛИТИКА

Сборник сценариев и тренажер по расчету основных метрик

ВНУТРИ ЭТОГО ВЫПУСКА:

Гигиена данных
Анализ качества продукта
Экономика и монетизация
Сегментация аудитории
и многое другое

Автор Kanstantsin Hupalau

Оглавление

- **Аналитика здравого смысла** **4**
- **Как развернуть тестовое окружение** **5**
- **О датасетах и зерне данных** **7**
- **Описание датасетов** **8**
- **Справочник метрик и терминов** **9**
- **Глава 1. Гигиена данных** **11**
- Поиск пропущенных связей 12
- Поиск дубликатов в справочнике 14
- Двойные списания 16
- **Глава 2. Базовые метрики роста** **18**
- Дневная аудитория (DAU) 19
- Месячная аудитория (MAU) 21
- Динамика регистраций 23
- Дневная и накопительная выручка 25
- Net Revenue - очистка от налогов 27
- Sticky Factor - оценка вовлеченности 29
- **Глава 3. Анализ качества продукта** **31**
- Время до покупки 32
- Скользящее среднее 34
- Анализ выходного дня 36
- Тепловая карта 38
- **Глава 4. Экономика и монетизация** **40**
- ARPU vs. ARPPU 41
- Структура выручки: New vs Recurring 43
- Продлеваемость подписок 45
- Анализ ценовых точек 47
- Поиск «Китов» 49

Оглавление

- **Глава 5. Сегментация** **51**
- Категорийная сегментация 52
- Поведенческая сегментация 54
- RFM-скоринг 56

- **Глава 6. A/B Тестирование** **58**
- Конверсия в покупку и расчет Uplift 59
- Сравнение среднего чека 61
- Кумулятивная выручка по группам 63

- **ГЛАВА 7. Жизненный цикл** **65**
- Retention Day N 66
- Расчет оттока 68
- Накопительный LTV 70

- **Послесловие** **72**

Аналитика здравого смысла

Эта книга не про синтаксис SQL, синтаксис можно выучить с помощью документации. Задача этого учебника научить вас думать над бизнес-задачей применительно к анализу продуктовых данных.

В идеальном мире данные стерильны, документация актуальна, а графики всегда растут вверх. В реальном продакшене данные «грязные», логи двоят, а «взрыв данных» поджидает на каждом шаге. Поэтому продуктовый аналитик должен уметь видеть за строками кода живой продукт и бизнес-риски.

Три кита этого практикума

1. **Здоровый скептицизм.** Если ваш запрос выдал слишком красивую цифру — скорее всего, вы столкнулись с ошибкой в логике или техническим багом. Ищите дубли, проверяйте «зерно», сомневайтесь в джойнах. Хороший аналитик верит не цифре, а проверенному алгоритму.
2. **Метрики помогают принимать решения.** Мы не считаем LTV ради красивого отчета. Мы считаем его, чтобы понять, стоит ли нам закупать трафик в следующем месяце. Каждый запрос в этой книге отвечает на конкретный бизнес-вопрос и обусловлен бизнес-задачей.
3. **Читаемость важнее сложности.** Ваш код будут читать коллеги (и вы сами через три месяца). Мы используем CTE (WITH), понятные названия подзапросов и логичное форматирование не для красоты, а для минимизации ошибок. Сложный код — враг точности.

Пройдя этот путь, вы освоитесь в базовых методах продуктового анализа в мобильном приложении с подписочной моделью, что приведет вас на шаг ближе к статусу настоящего интеллектуального партнера для бизнеса. Будьте критичны к своим результатам, внимательны к деталям и не забывайте про здравый смысл.

Данные никогда не врут, но они часто пытаются нас запутать.

Kanstantsin Hupalau

Как развернуть тестовое окружение

Чтобы запустить запросы из этой книги, вам понадобится песочница в Google BigQuery. Это бесплатно (в рамках sandbox-лимитов) и не требует привязки карты.

Шаг 1. Получение материалов

Перейдите в репозиторий проекта на GitHub и скачайте архив с проектом. Вам понадобятся две папки:

1. /data: два CSV-файла с сырыми данными (users.csv и payments.csv).
2. /queries: все SQL-запросы из этой книги, разбитые по главам.

Шаг 2. Подготовка BigQuery

Зайдите в Google Cloud Console.

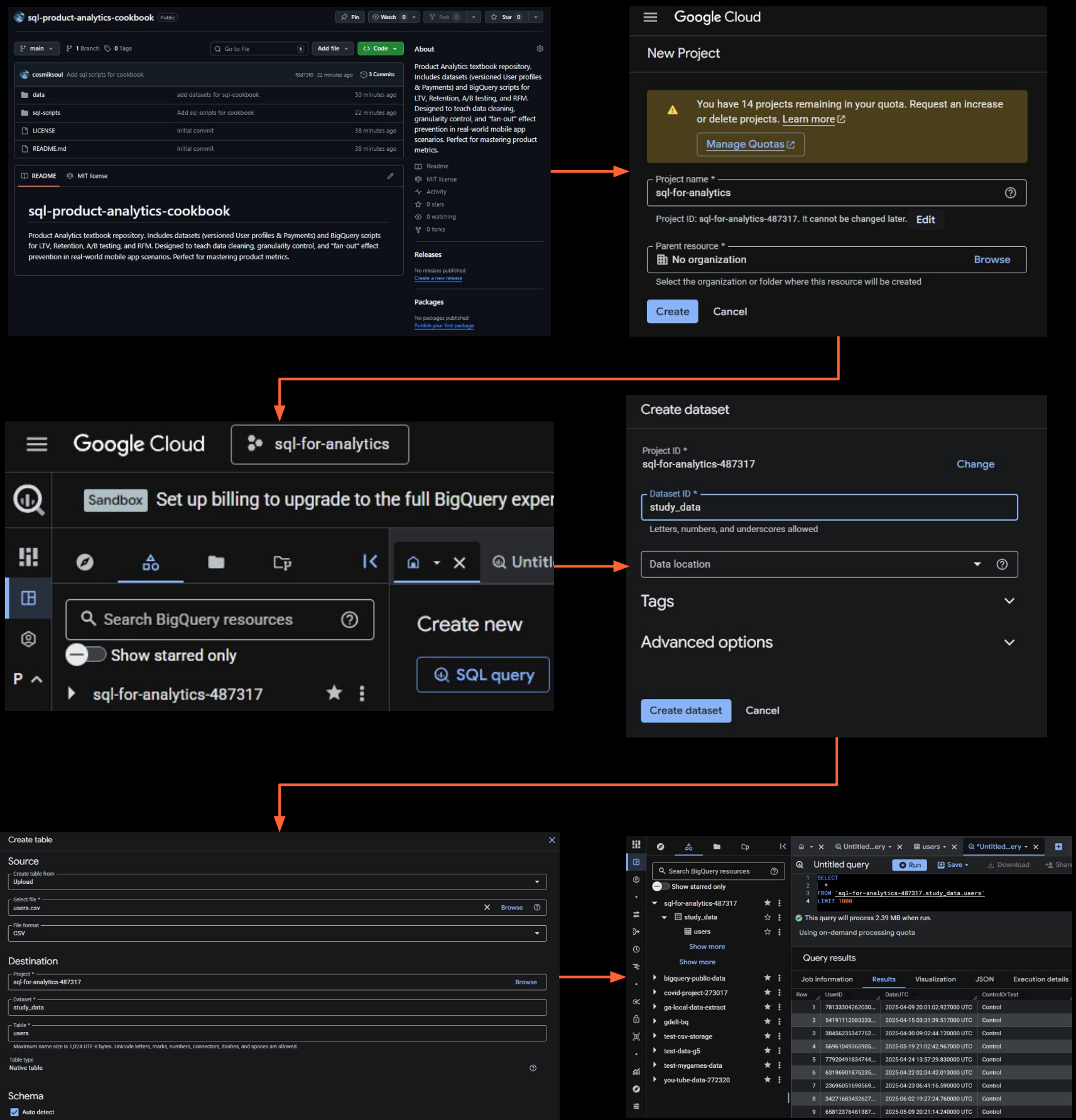
1. Создайте проект: Нажмите на выпадающее меню в левом верхнем углу -> New Project. Назовите его, например, sql-for-analytics.
2. Создайте датасет: В панели BigQuery нажмите на три точки рядом с ID вашего проекта и выберите Create dataset.
   - ID датасета: study_data.
   - Location: выберите ближайший к вам регион (например, EU или US).

Шаг 3. Загрузка данных

Повторите следующие действия для каждого из двух CSV-файлов:

1. Внутри датасета study_data нажмите Create Table.
2. Source: выберите Upload и выберите файл с компьютера.
3. File format: CSV.
4. Table name: назовите таблицы точно так же, как файлы: users и payments. Schema: поставьте галочку на Auto detect (BigQuery сам поймет, где дата, а где число).
5. В настройках Advanced options проверьте, чтобы в поле Header rows to skip установлено значение 1, чтобы первая строка (заголовки) не превратилась в данные.

Как развернуть тестовое окружение



О датасетах и зерне данных

Для обучения мы используем два датасета, имитирующих базу типичного мобильного дейтинг-приложения. В реальности данные могут быть распределены по десяткам таблиц (логины, атрибуция, клики), но мы используем упрощенную модель, чтобы сфокусироваться на логике запросов.

Зерно данных (data grain)

Перед написанием любого запроса определите зерно таблицы — это понимание того, что именно описывает одна её строка. Если зерна таблиц не совпадают, их нельзя объединять «в лоб».

Таблица Payments. Зерно — транзакция.
Здесь все просто: одна строка — это одна успешная оплата. Сумма по колонке RevenueUSD дает реальную выручку приложения.

Таблица Users. Зерно — версия состояния пользователя.
Здесь немного сложнее. Мы эмулируем механизм версионности. Каждое изменение в профиле (загрузка фото, смена статуса) не перезаписывает данные, а создает новую строку. Это требует особого внимания, так как один UserID может иметь 5–10 записей.

Ловушка «Взрыва данных»

Если вы сделаете JOIN платежа к справочнику users, где у пользователя может быть несколько (допустим, 5) записей, то произойдет следующее:

- База найдет один платеж на $10.
- Увидит 5 версий профиля этого юзера в таблице users.
- Дублирует этот платеж 5 раз, чтобы присоединить его к каждой строке из users.
- В отчете вместо $10 появится $50. Поздравляю, вы «создали» деньги из воздуха.

Золотые правила работы

Чтобы ваши цифры всегда бились с реальностью, следуйте трем правилам:

- Для поиска даты регистрации всегда используйте MIN(DateUTC).
- Для проверки актуальных свойств используйте последнюю по времени запись профиля.
- Для денежных метрик дедуплицируйте Users перед джойном через подзапрос или CTE. Иначе выручка в отчете может быть завышена

Описание датасетов

Таблица Users. Лог состояний профиля

Зерно данных: Состояние профиля в конкретный момент времени

Поле	Тип	Описание
UserID	STRING	Уникальный ID пользователя.
DateUTC	TIMESTAMP	Время создания записи (момент изменения профиля или регистрации).
ControlOrTest	STRING	Группа в A/B тесте. Не меняется со временем.
MaritalStatus	STRING	Текущий семейный статус на указанную дату.
PhotoCount	INT	Количество загруженных фото на указанную дату.

Таблица Payments. Транзакции

Зерно данных: Одна успешная оплата

Поле	Тип	Описание
PaymentAttemptID	STRING	Уникальный ID платежа.
UserID	STRING	Связь с профилем пользователя.
DateUTC	TIMESTAMP	Время платежа.
RevenueUSD	FLOAT	Сумма (выручка).
RenewalType	STRING	`New` — первая покупка, `Renewal` — автоматическое продление.

Справочник метрик и терминов

- **Sanity Check** — базовые тесты данных на логичность, отсутствие дублей и критических багов логирования.
- **Missing Join (Orphans)** — «сиротские» записи. Ситуация, когда в таблице фактов (например, платежи) есть данные, для которых нет соответствующих записей в справочниках (пользователях).
- **Double Charges** — двойные списания. Технический баг, при котором одна транзакция дублируется в базе, искажая выручку в большую сторону.
- **DAU / MAU (Daily / Monthly Active Users)** — количество уникальных пользователей за день или месяц.
- **Sticky Factor (DAU/MAU)** — коэффициент «липкости». Показывает, насколько регулярно пользователи возвращаются в продукт. Чем выше %, тем ближе продукт к ежедневной привычке.
- **Retention Day N** — удержание. Процент пользователей, вернувшихся в продукт ровно на N-й день после регистрации.
- **Churn Rate** — коэффициент оттока. Доля пользователей, которые перестали пользоваться продуктом или продлевать подписку за определённый период.
- **ARPU (Average Revenue Per User)** — средний доход на одного активного пользователя. Считается как вся выручка, делённая на всю активную аудиторию (DAU или MAU).
- **ARPPU (Average Revenue Per Paying User)** — средний доход на одного платящего пользователя. Считается только по тем, кто совершил покупку.
- **Net Revenue** — чистая выручка после вычета налогов (VAT/GST) и комиссий сторов (App Store / Google Play). Именно эти деньги «доходят» до бизнеса.
- **Cumulative Revenue** — накопительная выручка. Сумма всех доходов с начала периода или с момента регистрации когорты.
- **New vs Recurring Revenue** — разделение выручки на «новую» (первые покупки) и «рекуррентную» (продления подписок). Показывает устойчивость финансовой модели.

Справочник метрик и терминов

- **Renewal Rate** — процент продлений. Доля пользователей, которые перешли с предыдущего периода подписки на следующий.
- **Cohort** — группа пользователей, которые совершили определенное действие в один и тот же промежуток времени (например, зарегистрировались в марте 2026 года).
- **Whales («Киты»)** — сегмент сверхплатящих пользователей (обычно 1–2% базы), которые приносят значительную долю выручки.
- **Segmentation** — разделение аудитории на группы по определенным признакам: географии, платформе, поведению или демографии.
- **Bins** — «корзины». Метод группировки непрерывного числа (например, возраст или количество фото) в дискретные категории (например, «0 фото», «1–3 фото», «4+ фото»).
- **RFM-скоринг** — метод сегментации по трём метрикам: Recency (давность покупки), Frequency (частота), Monetary (сумма). Помогает отличить лояльных фанатов от «спящих» клиентов.
- **Time to Convert** — окно конверсии. Время, которое проходит с момента регистрации (или другого триггера) до совершения первой покупки.
- **Moving Average** — скользящее среднее. Метод сглаживания графиков, чтобы убрать «шум» и увидеть реальный тренд за рамками дневных колебаний.
- **Control & Test** — контрольная группа (без изменений) и тестовая группа (с новой фичей), на которых проверяется гипотеза в рамках теста.
- **Uplift** — относительный прирост метрики в тестовой группе по сравнению с контрольной. Показывает эффективность изменений.
- **Mean, Variance, StdDev** — среднее, дисперсия и стандартное отклонение. Набор метрик, необходимых для оценки распределения данных и проведения статистических тестов (например, T-test).
- **Convergence** — сходимость. Состояние A/B теста, когда метрики в группах стабилизируются, а накопленный результат перестает хаотично прыгать.
- **Зерно данных (data grain)** - уровень детализации или степень атомарности данных, что именно описывает ОДНА строка в конкретной таблице таблице

Глава 1. Гигиена данных

Прежде чем считать сложные агрегаты или делать далеко идущие выводы, нужно убедиться, что наш «фундамент» не плывет. В этой главе мы занимаемся проверкой наших датасетов: ищем технические баги и логические разрывы, которые могут превратить любой красивый отчет в дезинформацию.

Что вас ждет внутри главы

В этой главе собраны рецепты скриптов для проверки целостности данных: поиск «сиротских» записей без связей, аудит справочников на предмет дубликатов и детектор технических аномалий в транзакциях. Эти проверки позволяют выявить ошибки логирования и баги платежных систем до того, как они попадут в бизнес-отчеты.

Зачем это в работе

Все очень просто, Garbage In - Garbage Out. Мы не хотим дать бизнесу ложную или искаженную ошибкой информацию для принятия ключевых решений о продукте. Если в системе есть двойные списания или платежи без привязки к пользователю, ваш ARPU будет завышен, а стоимость привлечения (CAC) — рассчитана неверно. Эти запросы — ваш «входной фильтр» перед любой серьезной задачей.

Что осталось за кадром

Мы не будем рассматривать сложные методы, связанные с дата инжинирингом, такие как глубокий аудит логирования и Data Lineage (отслеживание пути данных от фронтенда до БД). И в целом, здесь мы скорее фиксируем последствия багов в таблицах, а не лечим их причины.

Что почитать самостоятельно:

- Понятие «Целостности данных» (Entity Integrity / Referential Integrity)
- Data Quality Dimensions (Completeness / Consistency / Accuracy)
- ACID и принципы нормализации баз данных (1NF / 2NF / 3NF)
- Концепция Idempotency (Data Deduplication / Retry Logic)
- Окна для дедупликации (ROW_NUMBER / PARTITION BY)

Поиск пропущенных связей

Задача: Выявить нарушение целостности данных, проверив, для всех ли транзакций существует валидный профиль пользователя в справочнике. Это необходимо для исключения ошибок при расчете LTV и ROI, где требуется объединение таблиц.

Уровень: Начинающий
Что тренируем: *LEFT JOIN для дебаггинга, фильтрация по IS NULL.*

Технический разбор

1. **Источник данных (FROM):** Загружаем таблицу платежей p как основу выборки.
2. **Обогащение (LEFT JOIN):** Присоединяем таблицу пользователей u по ключу UserID. Используем LEFT JOIN, чтобы сохранить все платежи, даже если пользователь не найден. В строках без совпадений поля из таблицы u будут заполнены NULL.
3. **Фильтрация (WHERE):** Отсекаем валидные записи, оставляя только те строки, где u.UserID оказался NULL (связь не найдена).
4. **Агрегация (SELECT):** Подсчитываем количество уникальных проблемных ключей и формируем список примеров для передачи инженерам.

Совет

В идеальном мире этот запрос возвращает ноль строк. Если он вернул данные, не спешите их удалять — часто это симптом поломки GDPR-скриптов (пользователя удалили по запросу, а платежи забыли анонимизировать) или сбоя в атрибуции (покупка пришла раньше регистрации). Если вы проигнорируете этих «сирот», ваш отчет по LTV и ROI будет занижен: деньги в кассе есть, а рекламного источника у них нет. Сначала почините данные на уровне инженерии, и только потом стройте графики.

Поиск пропущенных связей

Логика запроса

Мы берем таблицу платежей и пытаемся прикрепить к каждой транзакции профиль пользователя. Если база данных не находит пару (возвращает пустоту), мы оставляем такую транзакцию в списке. В конце мы считаем, сколько таких «бесхозных» платежей нашлось.

Пример кода

```sql
SELECT
  -- Считаем, сколько уникальных юзеров есть в платежах,
  -- но нет в таблице регистраций
  COUNT(DISTINCT p.UserID) AS users_missing_in_joins,
  -- Собираем 5 примеров ID в один список
  ARRAY_AGG(DISTINCT p.UserID LIMIT 5) AS example_ids
FROM `project.dataset.payments` AS p
  -- Присоединяем таблицу `users`
LEFT JOIN `project.dataset.users` AS u
  ON p.UserID = u.UserID
WHERE u.UserID IS NULL;
```

Ожидаемый результат

Query results

Job information | Results | Visualization

Row	users_missing_in...	example_ids
1	0	*0 rows*

Поиск дубликатов в справочнике

Задача: Проверить таблицу пользователей на уникальность первичных ключей (UserID). Наличие дубликатов приводит к кратному завышению метрик (например, выручки) при объединении (JOIN) с таблицей транзакций.

Уровень: Начинающий
Что тренируем: *Поиск коллизий через GROUP BY, отличие HAVING от WHERE при фильтрации агрегированных данных.*

Технический разбор

1. **Группировка (GROUP BY):** Схлопываем таблицу по полю UserID, чтобы анализировать каждый идентификатор как единую сущность.
2. **Подсчет (COUNT):** Для каждого UserID вычисляем количество строк, в которых он встречается.
3. **Фильтрация агрегатов (HAVING):** Отбираем только те группы, где количество строк строго больше 1. Мы используем HAVING вместо WHERE, так как фильтрация происходит уже после выполнения группировки.
4. **Сортировка (ORDER BY):** Выводим самые частые дубликаты вверху списка для приоритетного разбора.

Совет

Если вы нашли дубликаты, стандартная практика «чистки» — оставить запись с самой ранней датой регистрации (MIN(DateUTC)), считая остальные техническим сбоем (например, дабл-клик по кнопке). Однако полагаться на DISTINCT в каждом аналитическом запросе — плохая привычка, которая маскирует проблему и замедляет базу данных. Лучше создайте новую, чистую материализованную таблицу, устранив дубликаты физически один раз и навсегда.

Поиск дубликатов в справочнике

Логика запроса

Мы группируем таблицу пользователей по их уникальному ID и пересчитываем количество строк в каждой группе. Если где-то счетчик показывает больше 1, значит, пользователь был записан в базу несколько раз. Мы выводим только эти проблемные ID.

Пример кода

```sql
SELECT
  u.UserID,
  -- Подсчитываем число вхождений ID (должно быть равно 1)
  COUNT(*) as duplicate_count
FROM `project.dataset.users` AS u
GROUP BY 1
-- Оставляем только те ID, которые встречаются более одного раза
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC
LIMIT 10;
```

Ожидаемый результат

Job information | Results | Visualization

Row	UserID	duplicate_count
1	24136667144205...	26
2	73657631086203...	18
3	59668682955108...	17
4	90563302473427...	17
5	92083720957130...	16
6	25190945866887...	15
7	81358956708621...	12
8	11989464894041...	10

Двойные списания

Задача: Идентифицировать пользователей, у которых произошли повторные списания одной и той же суммы за аномально короткий промежуток времени (менее 60 секунд). Такие паттерны указывают на техническую ошибку биллинга.

Уровень: Продвинутый
Что тренируем: *Оконные функции (LEAD) для сравнения соседних строк, вычисление временных интервалов (TIMESTAMP_DIFF), использование CTE для читаемости кода.*

Технический разбор

1. **Подготовка данных (CTE):** Создаем временную таблицу PaymentChain, чтобы предварительно обработать поток платежей.
2. **Оконная функция (LEAD):** Для каждой строки получаем дату (next_date) и сумму (next_rev) следующего платежа этого же пользователя.
   - **PARTITION BY UserID:** Ограничиваем область видимости одним юзером.
   - **ORDER BY DateUTC:** Гарантируем хронологический порядок
3. **Вычисление интервала (TIMESTAMP_DIFF):** В основном запросе считаем разницу в секундах между текущим платежом и следующим (next_date).
4. **Фильтрация инцидентов (WHERE):** Оставляем только те пары, где суммы совпадают, а временной интервал меньше порога (60 секунд).

Совет

Порог в 60 секунд выбран эмпирически, но для разных шлюзов он может отличаться. Важный нюанс: иногда платежная система делает автоматический retry (повтор) через секунду после ошибки. Это нормально, если первая транзакция была неуспешной. Поэтому обязательно добавьте в выборку проверку статуса: WHERE status = 'success'. Двойные списания могут привести к диспутам и штрафам от платежных систем.

Двойные списания

Логика запроса

Мы выстраиваем платежи каждого пользователя в хронологическую цепочку. Затем для каждой транзакции мы «подглядываем» в следующую. Если сумма совпадает, а разница во времени меньше минуты — мы помечаем это как баг двойного списания.

Пример кода

```sql
WITH PaymentChain AS (
  SELECT
    p.UserID,
    p.DateUTC,
    p.RevenueUSD,
    -- Получаем данные о СЛЕДУЮЩЕМ платеже пользователя в той же строке
    LEAD(p.DateUTC) OVER (PARTITION BY p.UserID ORDER BY p.DateUTC) as next_date,
    LEAD(p.RevenueUSD) OVER (PARTITION BY p.UserID ORDER BY p.DateUTC) as next_rev
  FROM `project.dataset.payments` AS p
)

SELECT
  UserID,
  RevenueUSD,
  DateUTC as charge_1_time,
  next_date as charge_2_time,
  -- Вычисляем дельту времени в секундах
  TIMESTAMP_DIFF(next_date, DateUTC, SECOND) as diff_seconds
FROM PaymentChain
WHERE
  -- Условие бага: совпадают суммы И прошло мало времени
  RevenueUSD = next_rev
  AND TIMESTAMP_DIFF(next_date, DateUTC, SECOND) < 60
ORDER BY DateUTC DESC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	UserID	RevenueUSD	charge_1_time	charge_2_time	diff_seconds
1	39281889897889...	12.32	2025-08-30 01:26:30.420000 UTC	2025-08-30 01:27:26.960000 UTC	56
2	80613636150907...	108.28	2025-08-28 04:28:46.133000 UTC	2025-08-28 04:28:46.347000 UTC	0
3	42266656763339...	38.8	2025-08-27 17:00:45.560000 UTC	2025-08-27 17:00:47.803000 UTC	2
4	42266656763339...	38.8	2025-08-27 17:00:41.800000 UTC	2025-08-27 17:00:45.560000 UTC	3
5	42266656763339...	38.8	2025-08-27 17:00:39.467000 UTC	2025-08-27 17:00:41.800000 UTC	2
6	42266656763339...	38.8	2025-08-27 17:00:36.137000 UTC	2025-08-27 17:00:39.467000 UTC	3
7	68354224652705...	52.0	2025-08-27 01:29:54.710000 UTC	2025-08-27 01:29:54.793000 UTC	0

Глава 2. Базовые метрики роста

В этой главе мы переходим от проверки качества данных к джентельменскому набору аналитика - расчету «пульса» продукта. Мы учимся извлекать главные цифры, которые каждое утро хочет видеть руководство в своих дашбордах. Это основа мониторинга, без которой невозможно оценить масштаб бизнеса и динамику его развития.

Что вас ждет внутри главы

Расчеты ключевой активности (DAU, MAU) и продуктовой «липкости» (Sticky Factor), отслеживание темпов роста через динамику регистраций и выручки, а также замер эффективности через ежедневную конверсию. Особое внимание уделено «приземлению» данных к реальности: очистке выручки от налогов для получения честного Net Revenue.

Зачем это в работе

Чтобы видеть тренд. Базовые метрики позволяют быстро понять: растем мы или стагнируем. Накопительный итог (Cumulative Revenue) покажет реальную скорость наполнения бюджета, а Sticky Factor ответит на вопрос, становится ли сервис ежедневной привычкой или пользователи заходят в него раз в месяц. Это «первая линия обороны» аналитика против неопределенности.

Что осталось за кадром

Детализация и сегментация. Мы смотрим на продукт как на единое целое. Разбор по странам, девайсам или источникам трафика начнется в следующих главах. Также здесь мы работаем в календарном времени, не учитывая «возраст» пользователя (когортную логику).

Что почитать самостоятельно:

- Rolling Metrics (Rolling 30D MAU / Trailing 12M Revenue)
- Growth Accounting (New / Retained / Resurrected / Churned)
- Экономика сторов (Apple & Google Commissions / VAT & GST Taxes)
- Бенчмарки вовлеченности (Power User Curve / L5+ Usage)
- Эффект сезонности и праздников (Year-over-Year / Week-over-Week / Holiday Spikes)

Дневная аудитория (DAU)

Задача: Измерить ежедневную нагрузку на продукт. DAU (Daily Active Users) показывает количество уникальных пользователей, открывавших приложение в конкретный день. Метрика служит «пульсом» продукта: мгновенно реагирует на технические сбои или виральные эффекты.

Уровень: Начинающий
Что тренируем: *Агрегацию по времени (GROUP BY), дедупликацию (COUNT DISTINCT), работу с датами (DATE).*

Технический разбор

1. **Источник данных (FROM):** Используем таблицу событий (events), где записано каждое действие пользователя.
2. **Очистка даты (DATE):** Исходный timestamp события содержит время. Функция DATE() отсекает часы и минуты, оставляя календарный день для корректной группировки.
3. **Дедупликация (COUNT DISTINCT):** Критический шаг. Один пользователь может совершить 50 действий за день, но в DAU он должен попасть как одна единица. Без DISTINCT мы посчитаем сессии, а не людей.
4. **Сортировка (ORDER BY):** Выводим данные от свежих к старым, чтобы видеть актуальную картину.

Совет

Это упрощенный пример. Мы используем таблицу users как прокси активности, но в реальности DAU считают по таблице событий (events). Важно понимать, что просто открытие приложения (app_open/session_start) может быть слабым сигналом активности. Возможно вместо обычного DAU имеет смысл рассчитать Quality DAU — пользователей, которые совершили хотя бы одно значимое действие (например, view_item или message_send). Также всегда фильтруйте события background_fetch, когда приложение обновляется в фоне — это не активность пользователя.

Дневная аудитория (DAU)

Логика запроса

Мы игнорируем точное время активности, оставляя только дату. Затем собираем всех пользователей, заходивших в этот день, и считаем их количество. Главное условие — каждый человек считается только один раз, даже если он открывал приложение десять раз за день.

Пример кода

```sql
SELECT
  -- Превращаем '2025-05-12 14:30:00' в '2025-05-12'
  DATE(DateUTC) AS activity_date,
  -- Считаем уникальных пользователей
  COUNT(DISTINCT UserID) AS active_users
FROM `project.dataset.users` -- Таблица прокси-активности
GROUP BY activity_date
ORDER BY activity_date DESC;
```

Ожидаемый результат

Job information | Results | Visualization

Row	activity_date	active_users
1	2025-08-31	190
2	2025-08-30	346
3	2025-08-29	345
4	2025-08-28	356
5	2025-08-27	380
6	2025-08-26	377
7	2025-08-25	413
8	2025-08-24	401
9	2025-08-23	416
10	2025-08-22	372
11	2025-08-21	371

Месячная аудитория (MAU)

Задача: Оценить масштаб продукта. MAU (Monthly Active Users) показывает охват — сколько уникальных людей воспользовались сервисом хотя бы раз за календарный месяц. Это ключевая метрика для оценки доли рынка и отчетов перед инвесторами.

Уровень: Начинающий
Что тренируем: *Усечение дат (DATE_TRUNC), агрегацию на широких окнах.*

Технический разбор

1. **Округление даты (DATE_TRUNC):** Функция приводит любую дату к первому числу месяца (например, 2025-08-27 превращается в 2025-08-01). Это позволяет собрать все события месяца в одну группу.
2. **Агрегация (COUNT DISTINCT):** Пользователь, заходивший каждый день (30 раз за месяц), в метрике MAU учитывается только один раз.
3. **Группировка (GROUP BY):** Схлопываем данные по полю activity_month.

Совет

Сумма 30 значений DAU всегда будет больше, чем одно значение MAU, так как в DAU одни и те же люди дублируются изо дня в день. Для утилит (такси, заказ еды) MAU часто важнее DAU, так как паттерн использования не подразумевает ежедневного входа. MAU — очень инертная метрика. Если вы «сломаете» приложение в начале месяца, MAU отреагирует на это только через 30 дней (пользователи просто не успеют «оттечь» из статистики). Поэтому для оперативного управления MAU почти бесполезна — используйте её только для стратегического планирования и оценки емкости рынка.

Месячная аудитория (MAU)

Логика запроса

Мы «откатываем» любую дату активности к первому числу месяца (например, все дни августа становятся «1 августа»). Это позволяет собрать всех пользователей в месячные группы. Как и в DAU, мы считаем каждого человека строго один раз за весь месяц.

Пример кода

```sql
SELECT
  -- Приводим дату к началу месяца
  DATE_TRUNC(DATE(DateUTC), MONTH) AS activity_month,
  -- Считаем уникальных пользователей за весь месяц
  COUNT(DISTINCT UserID) AS monthly_active_users
FROM `project.dataset.users`
GROUP BY activity_month
ORDER BY activity_month DESC;
```

Ожидаемый результат

Job information | Results | Visualization

Row	activity_month	monthly_active_users
1	2025-08-01	11687
2	2025-07-01	12067
3	2025-06-01	11001
4	2025-05-01	11750
5	2025-04-01	11973
6	2025-03-01	203
7	2025-02-01	56
8	2025-01-01	46
9	2024-12-01	36
10	2024-11-01	31
11	2024-10-01	27

Динамика регистраций

Задача: Отследить приток новых пользователей. Просто группировать активность по дням нельзя — мы смешаем новичков и старичков. Необходимо сначала найти дату первого появления («день рождения») для каждого пользователя.

Уровень: Средний
Что тренируем: *Использование CTE (Common Table Expressions), агрегатную функцию MIN(), логику «First Seen».*

Технический разбор

1. **Подготовка (CTE FirstSeen):** Группируем таблицу пользователей по UserID и находим минимальную дату (MIN) для каждого. Это и есть дата регистрации.
2. **Основной запрос (SELECT):** Обращаемся к подготовленной виртуальной таблице FirstSeen.
3. **Группировка (GROUP BY):** Считаем количество пользователей, у которых дата регистрации (first_date) совпадает с анализируемым днем.
4. **Сортировка (ORDER BY):** Упорядочиваем по дате регистрации.

Совет

Паттерн MIN(Date) GROUP BY ID — фундамент когортного анализа, но он требователен к ресурсам на больших данных. В продакшене (BigQuery) дешевле использовать готовые события от MMP (AppsFlyer/Adjust), такие как first_install или app_install. Они уже дедуплицированы на уровне устройства. Однако, если вам нужно найти дату регистрации именно аккаунта (а не установки приложения), то расчет через MIN(created_at) по базе бэкенда остается единственно верным способом, так как пользователь мог переустановить приложение 5 раз.

Динамика регистраций

Логика запроса

Прежде чем считать новичков, мы находим «день рождения» каждого аккаунта: просматриваем всю историю и выбираем самую раннюю дату (MIN) для каждого ID. Только после того, как у каждого пользователя определена одна дата регистрации, мы группируем их по дням.

Пример кода

```sql
WITH FirstSeen AS (
  -- Шаг 1: Находим дату первой активности для каждого юзера
  SELECT
    UserID,
    MIN(DateUTC) as first_date
  FROM `project.dataset.users`
  GROUP BY UserID
)

SELECT
  -- Шаг 2: Группируем новичков по дате первого появления
  DATE(first_date) as join_date,
  COUNT(UserID) as new_users
FROM FirstSeen
GROUP BY join_date
ORDER BY join_date DESC;
```

Ожидаемый результат

Job information | Results | Visualization

Row	join_date	new_users
1	2025-08-31	190
2	2025-08-30	346
3	2025-08-29	345
4	2025-08-28	356
5	2025-08-27	380
6	2025-08-26	377
7	2025-08-25	413
8	2025-08-24	401
9	2025-08-23	416

Дневная и накопительная выручка

Задача: Показать финансовый прогресс. Дневная выручка волатильна, а график накопительного итога (Cumulative Revenue) позволяет увидеть общий тренд роста бизнеса и быстрее заметить замедление («выход на плато»).

Уровень: Средний
Что тренируем: *Оконные функции с нарастающим итогом (SUM() OVER), фреймы строк (UNBOUNDED PRECEDING).*

Технический разбор

1. **Предварительный расчет (CTE):** Считаем обычную сумму выручки за каждый день (daily_rev) с помощью простой группировки.
2. **Оконная функция (OVER):** В основном запросе применяем функцию суммы поверх уже сгруппированных строк.
3. **Определение окна (ORDER BY date):** Указываем, что данные нужно обрабатывать строго в хронологическом порядке.
4. **Накопление (ROWS BETWEEN...):** Инструкция UNBOUNDED PRECEDING AND CURRENT ROW говорит базе: «Для текущей строки сложи выручку за этот день и за все предыдущие дни с начала времен».

Совет

График накопительного итога (Cumulative) полезен не только для отчетности, но и для сравнения темпов роста (Velocity). Наложите на одном графике линии накопительной выручки за текущий и прошлый месяцы (сопоставив День 1 к Дню 1). Если линия текущего месяца идет круче вверх — значит, вы растете быстрее, чем в прошлом периоде. Если линии пересеклись и текущая пошла ниже — вы начали отставать от плана. Любое горизонтальное плато на этом графике — сигнал тревоги (продажи встали).

Дневная и накопительная выручка

Логика запроса

Сначала мы считаем обычную выручку за каждый день. А для накопительного графика мы говорим базе: «Для каждой даты возьми выручку этого дня и прибавь к ней суммы всех предыдущих дней с самого начала». Так получается постоянно растущая линия общего дохода.

Пример кода

```sql
WITH DailyRevenue AS (
  SELECT
    DATE(DateUTC) as date,
    SUM(RevenueUSD) as daily_rev
  FROM `project.dataset.payments`
  GROUP BY date
)
SELECT
  date,
  daily_rev,
  -- Считаем сумму с накоплением
  SUM(daily_rev) OVER (
    ORDER BY date ASC
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) as cumulative_revenue
FROM DailyRevenue
ORDER BY date DESC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON

Row	date	daily_rev	cumulative_reven...
1	2025-08-31	697.1100000000...	181783.03
2	2025-08-30	1447.41	181085.92
3	2025-08-29	1466.569999999...	179638.51
4	2025-08-28	1448.289999999...	178171.94
5	2025-08-27	1304.419999999...	176723.65
6	2025-08-26	712.3100000000...	175419.23
7	2025-08-25	1388.529999999...	174706.92
8	2025-08-24	1727.15	173318.39
9	2025-08-23	1178.760000000...	171591.24
10	2025-08-22	675.7499999999...	170412.48

Net Revenue - очистка от налогов

Задача: Рассчитать реальные деньги, которые поступят на счет компании, очистив Gross Revenue от комиссии стора (Apple/Google) и налогов (VAT/GST).

Уровень: Начинающий
Что тренируем: *Арифметику внутри агрегаций, округление (ROUND), усечение дат (DATE_TRUNC).*

Технический разбор

1. **Группировка (DATE_TRUNC):** Сводим данные по месяцам, так как финансовые сверки со сторами обычно происходят раз в месяц.
2. **Вычет комиссии (* 0.7):** Эмулируем удержание 30% платформой (умножаем на 0.7, чтобы получить остаток).
3. **Вычет налога (/ 1.2):** «Вынимаем» НДС (20%) из суммы. Важно делить на 1 + ставка, а не вычитать проценты, так как налог обычно уже включен в цену для пользователя (Gross Price).
4. **Округление (ROUND):** Ограничиваем результат двумя знаками после запятой.

Совет

Помимо комиссий и налогов, в реальном биллинге есть третья проблема — возвраты (Refunds). Они могут приходить с задержкой в 30-60 дней. Если в вашей таблице payments возвраты просто удаляются, этот скрипт покажет завышенную выручку за прошлые месяцы. Правильная практика — записывать возврат как новую строку с отрицательной суммой (RevenueUSD = -9.99). Убедитесь, что ваш запрос корректно суммирует эти минусы. И никогда не хардкодьте налоги — создайте справочник dict_taxes и джойните его по CountryCode.

Net Revenue - очистка от налогов

Логика запроса

Мы суммируем выручку по месяцам, а затем внутри формулы применяем арифметику: умножаем на 0.7, чтобы убрать комиссию магазина, и делим на 1.2, чтобы «вынуть» НДС, который уже сидит внутри цены. В итоге получаем чистые деньги бизнеса.

Пример кода

```sql
SELECT
  DATE_TRUNC(DATE(DateUTC), MONTH) as month,
  SUM(RevenueUSD) as gross_revenue,
  -- Вычитаем комиссию стора (30%)
  ROUND(SUM(RevenueUSD) * 0.7, 2) as net_after_store_fee,
  -- Вычитаем комиссию и налог (VAT 20%)
  ROUND(SUM(RevenueUSD) * 0.7 / 1.2, 2) as net_revenue_final
FROM `project.dataset.payments`
GROUP BY month
ORDER BY month DESC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details

Row	month	gross_revenue	net_after_store_fee	net_revenue_final
1	2025-08-01	39493.69000000...	27645.58	23037.99
2	2025-07-01	40143.32000000...	28100.32	23416.94
3	2025-06-01	35730.00000000...	25011.0	20842.5
4	2025-05-01	34100.91999999...	23870.64	19892.2
5	2025-04-01	32149.33999999...	22504.54	18753.78
6	2025-03-01	165.76	116.03	96.69

Sticky Factor - оценка вовлеченности

Задача: Оценить «липкость» продукта (Habit-forming potential). Рассчитывается как отношение DAU к MAU. Если Sticky Factor = 20%, значит, средний пользователь открывает приложение 6 дней в месяц (20% от 30 дней).

Уровень: Продвинутый
Что тренируем: *Сложные джойны по диапазону дат (BETWEEN), концепцию скользящего окна (Rolling Window).*

Технический разбор

1. **Расчет DAU (CTE DailyStats):** Считаем количество уникальных пользователей для каждого дня.
2. **Расчет скользящего MAU (CTE MonthlyStats):** Для каждой даты рассчитываем количество уникальных пользователей за последние 30 дней (Rolling 30-day Window).
3. **Соединение (JOIN ON BETWEEN):** Используем условие BETWEEN, чтобы привязать к каждой текущей дате интервал в 29 предыдущих дней.
4. **Финальный расчет (SELECT):** Делим дневную аудиторию на месячную и переводим в проценты.

Совет

Эталоны метрики сильно зависят от ниши приложения: для соцсетей нормой считается 50%+, для игр — 10–20%, для сервисов бронирования — менее 5%. Однако будьте осторожны, Sticky Factor можно искусственно «накрутить» агрессивными пуш-уведомлениями, заставляя пользователей заходить в приложение без реальной пользы. Если DAU/MAU растет, а средняя длительность сессии падает — возможно, вы просто раздражаете аудиторию, и скоро может увеличиться отток. Смотрите на эти метрики только в паре.

Sticky Factor - оценка вовлеченности

Логика запроса

Для каждого конкретного дня мы вычисляем два числа: сколько людей пришло сегодня (DAU) и сколько уникальных людей было за последние 30 дней (Rolling MAU). Деление первого на второе показывает, какой процент месячной аудитории пользуется продуктом прямо сейчас.

Пример кода

```sql
WITH DailyStats AS (
  SELECT
    DATE(DateUTC) as activity_date,
    COUNT(DISTINCT UserID) as dau
  FROM `project.dataset.users`
  GROUP BY activity_date
),
MonthlyStats AS (
  SELECT
    d.activity_date,
    -- Скользящее MAU (пользователи за последние 30 дней относительно текущей даты)
    COUNT(DISTINCT u.UserID) as rolling_mau
  FROM DailyStats d
  JOIN `project.dataset.users` u
    ON DATE(u.DateUTC) BETWEEN DATE_SUB(d.activity_date, INTERVAL 29 DAY) AND d.activity_date
  GROUP BY d.activity_date
)
SELECT
  d.activity_date,
  d.dau,
  m.rolling_mau,
  ROUND(d.dau / NULLIF(m.rolling_mau, 0) * 100, 1) as sticky_factor_pct
FROM DailyStats d
JOIN MonthlyStats m ON d.activity_date = m.activity_date
ORDER BY d.activity_date DESC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details

Row	activity_date	dau	rolling_mau	sticky_factor_pct
1	2025-08-31	190	11354	1.7
2	2025-08-30	346	11497	3.0
3	2025-08-29	345	11500	3.0
4	2025-08-28	356	11530	3.1
5	2025-08-27	380	11544	3.3

Глава 3. Анализ качества продукта

В этой главе мы углубимся в анализ поведения пользователей и перейдем от количественных показателей к качественным. Мы научимся анализировать не просто «сколько» действий совершено, а «как» они распределены во времени. Это позволит нам заглянуть в быт пользователя: как быстро наша аудитория принимает решение о покупке, в какие часы она наиболее активна и как отфильтровать лишний шум, чтобы увидеть реальный тренд.

Что вас ждет внутри главы

Инструменты для замера скорости принятия решений (Time to Convert), механики сглаживания волатильных графиков через скользящее среднее и методы подготовки данных для визуализации цикличной активности (тепловые карты и анализ выходных).

Зачем это в работе

Чтобы понимать «ритм» вашего продукта. Знание окна конверсии помогает выставить правильные сроки ожидания для маркетинговых кампаний (чтобы не паниковать раньше времени), а сглаживание трендов спасает от ложных выводов при резких дневных скачках выручки. Тепловые карты — это ваш план-график для рассылок и технических работ, основанный на реальном поведении, а не на догадках.

Что осталось за кадром

Предиктивная аналитика. Мы описываем то, что уже произошло, но пока не пытаемся предсказать, какой именно пользователь заплатит завтра. Также здесь мы не учитываем влияние внешних факторов (праздники, мировые события), которые могут искажать «здоровый» пульс продукта.

Что почитать самостоятельно:

- Виды скользящих средних (SMA / WMA / EMA)
- Анализ выживаемости (Survival Function / Hazard Rate / Метод Каплана-Мейера)
- Принципы построения гистограмм (Survival Function / Hazard Rate)
- Таймзоны и нормализация (ISO 8601 / UTC Offset)

Время до покупки

Задача: Понять, как быстро новые пользователи конвертируются в платящих. Строим гистограмму распределения времени от регистрации до первой оплаты. Это помогает оценить эффективность онбординга: если большинство покупок происходит в первый час — онбординг работает отлично (импульсивный спрос). Если через неделю — работает email-маркетинг.

Уровень: Средний
Что тренируем: *Работу с временными интервалами (TIMESTAMP_DIFF), категоризацию данных (CASE WHEN), CTE.*

Технический разбор

1. **Подготовка (CTE FirstPay):** Сначала находим дату первой транзакции для каждого пользователя. Используем MIN(DateUTC) с фильтром WHERE RenewalType = 'New', чтобы исключить продления подписок.
2. **Объединение (JOIN):** Соединяем таблицу пользователей u (время регистрации) с найденными первыми платежами fp по ключу UserID.
3. **Вычисление интервала (TIMESTAMP_DIFF):** Считаем разницу во времени между датой регистрации и датой покупки.
4. **Категоризация (CASE WHEN):** Разбиваем пользователей на временные интервалы (бакеты): «до 1 часа», «сутки», «неделя». Без этого шага мы получили бы тысячи уникальных значений времени, непригодных для анализа.
5. **Группировка (GROUP BY):** Считаем количество пользователей в каждом бакете.

Совет

В мобильных приложениях, особенно в играх и утилитах, «золотое окно» конверсии — это первые 15–60 минут. Если пользователь не купил сразу, вероятность покупки падает экспоненциально. Для сложных продуктов или SaaS нормальным считается цикл сделки в 7–14 дней. Ваша цель — сдвигать распределение в сторону меньших интервалов.

Время до покупки

Логика запроса

Мы берем момент регистрации пользователя и момент его первой оплаты, а затем вычисляем время, прошедшее между ними. Чтобы не анализировать тысячи уникальных значений, мы раскладываем пользователей по «корзинам»: купил за час, за сутки или за неделю.

Пример кода

```sql
WITH FirstPay AS (
  SELECT
    UserID,
    MIN(DateUTC) as first_pay_date
  FROM `project.dataset.payments`
  WHERE RenewalType = 'New' -- Только первая покупка
  GROUP BY UserID
)
SELECT
  -- Создаем временные корзины (buckets)
  CASE
    WHEN TIMESTAMP_DIFF(fp.first_pay_date, u.DateUTC, HOUR) <= 1 THEN '01. Fast (< 1 Hour)'
    WHEN TIMESTAMP_DIFF(fp.first_pay_date, u.DateUTC, HOUR) <= 24 THEN '02. Same Day (1-24 Hours)'
    WHEN TIMESTAMP_DIFF(fp.first_pay_date, u.DateUTC, DAY) <= 3 THEN '03. Days 1-3'
    WHEN TIMESTAMP_DIFF(fp.first_pay_date, u.DateUTC, DAY) <= 7 THEN '04. First Week'
    ELSE '05. Late Bloomers (> 7 Days)'
  END as time_to_convert,
  -- Подсчитываем количество конверсий (COUNT DISTINCT для надежности)
  COUNT(DISTINCT u.UserID) as conversion_count
-- Защита: берем MIN(DateUTC), если в логах регистрации есть дубли
FROM (SELECT UserID, MIN(DateUTC) as DateUTC FROM `project.dataset.users` GROUP BY 1) u
JOIN FirstPay fp ON u.UserID = fp.UserID
GROUP BY time_to_convert
ORDER BY time_to_convert ASC;
```

Ожидаемый результат

Job information | Results | Visualization

Row	time_to_convert	conversion_count
1	01. Fast (< 1 Hour)	1341
2	02. Same Day (1-24 Hours)	492
3	03. Days 1-3	303
4	04. First Week	152
5	05. Late Bloomers (> 7 Days)	521

Скользящее среднее

Задача: Очистить график от случайного шума. Дневные метрики (DAU, выручка) часто «скачут» из-за выходных или праздников. Скользящее среднее (обычно 7-дневное) сглаживает эти пики, показывая реальный тренд роста или падения продукта.

Уровень: Средний
Что тренируем: *Оконные функции с диапазоном (ROWS BETWEEN), фильтрацию шума.*

Технический разбор

1. **Агрегация по дням (CTE DailyStats):** Предварительно группируем исходные данные по датам, чтобы получить одну строку на каждый день.
2. **Определение окна (OVER):** В основном запросе задаем окно размером в 7 дней.
3. **Сортировка в окне (ORDER BY):** Обязательно указываем хронологический порядок, чтобы окно двигалось от прошлого к будущему.
4. **Расчет среднего (AVG):** Инструкция ROWS BETWEEN 6 PRECEDING AND CURRENT ROW берет текущее значение и 6 предыдущих, суммирует их и делит на 7. Полученная точка на графике является средним арифметическим за неделю.

Совет

Стандарт индустрии — 7 дней (или 28 для месячных трендов), так как это идеально гасит недельную сезонность. Окно в 5, 10 или 30 дней оставит на графике искусственные «волны». Важно помнить что скользящее среднее — это запаздывающий индикатор (Lagging Indicator). Если у вас сломалась оплата и выручка упала в ноль, 7-дневное среднее снизится лишь немного. Оно покажет реальный масштаб катастрофы только через 3–4 дня. Поэтому никогда не настраивайте алерты на скользящее среднее — используйте его только для анализа долгосрочных трендов.

Скользящее среднее

Логика запроса

Мы смотрим на данные через «окошко» шириной в 7 дней. Для каждой точки на графике мы суммируем результаты за эту неделю и делим на 7. Когда наступает новый день, окно сдвигается на шаг вперед, захватывая свежие данные и отбрасывая старые. Так мы получаем линию, очищенную от случайностей одного дня.

Пример кода

```sql
WITH DailyStats AS (
  SELECT
    DATE(DateUTC) as activity_date,
    COUNT(DISTINCT UserID) as daily_users
  FROM `project.dataset.users`
  GROUP BY activity_date
)
SELECT
  activity_date,
  daily_users,
  -- Среднее за текущий день + 6 предыдущих
  ROUND(AVG(daily_users) OVER (
    ORDER BY activity_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ), 1) as ma_7_day_users
FROM DailyStats
ORDER BY activity_date DESC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON

Row	activity_date	daily_users	ma_7_day_users
1	2025-08-31	190	343.9
2	2025-08-30	346	374.0
3	2025-08-29	345	384.0
4	2025-08-28	356	387.9
5	2025-08-27	380	390.0
6	2025-08-26	377	391.0
7	2025-08-25	413	390.1

Анализ выходного дня

Задача: Выяснить, в какие дни недели пользователи приносят больше денег или активности. Это критически важно для планирования маркетинговых бюджетов — нет смысла заливать трафик в понедельник, если ваша аудитория покупает только в пятницу вечером.

Уровень: Средний
Что тренируем: *Форматирование дат (FORMAT_DATE), извлечение дня недели (EXTRACT), кастомную сортировку.*

Технический разбор

1. **Форматирование (FORMAT_DATE):** Преобразуем дату в текстовое название дня недели (Monday, Tuesday...) для читаемости.
2. **Индекс сортировки (FORMAT_DATE('%u')):** Вычисляем порядковый номер дня (1=Mon ... 7=Sun), чтобы отсортировать график логически, а не по алфавиту.
3. **Группировка (GROUP BY):** Схлопываем данные по названию дня и его индексу.
4. **Агрегация (COUNT, SUM):** Считаем целевые метрики (транзакции, выручка) для каждого дня недели.

Совет

Обычно в B2B-продуктах пик активности приходится на вторник-четверг, а в развлекательных (игры, стриминг) — на пятницу и субботу. Если вы видите, что в выходные конверсия (Revenue / Users) выше, попробуйте перенести рассылку пуш-уведомлений именно на эти дни. Также эти данные можно использовать для оптимизации закупки трафика - можно, например, повышать ставки за клик в «горячие» дни и снижать в дни простоя. Это позволит выкупать самых ценных пользователей, не сливая бюджет в «мертвые» понедельники.

Анализ выходного дня

Логика запроса

Чтобы увидеть недельный ритм, мы «схлопываем» всю историю транзакций в 7 строк. Сначала превращаем конкретные даты в дни недели (Понедельник, Вторник...), затем суммируем выручку по каждой такой группе. В конце принудительно сортируем список от Понедельника к Воскресенью.

Пример кода

```
1  SELECT
2    -- Получаем название дня (Monday, Tuesday...)
3    FORMAT_DATE('%A', DateUTC) as day_name,
4    -- Считаем метрики
5    COUNT(PaymentAttemptID) as transactions,
6    ROUND(SUM(RevenueUSD), 0) as total_revenue
7  FROM `project.dataset.payments`
8  -- Группируем по дням недели
9  GROUP BY day_name, FORMAT_DATE('%u', DateUTC)
10 ORDER BY FORMAT_DATE('%u', DateUTC) ASC; -- Сортируем: 1=Mon ... 7=Sun
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details

Row	day_name	transactions	total_revenue
1	Monday	692	26192.0
2	Tuesday	619	21811.0
3	Wednesday	681	24267.0
4	Thursday	658	23851.0
5	Friday	704	25273.0
6	Saturday	733	30145.0
7	Sunday	769	30244.0

Тепловая карта

Задача: Подготовить данные для визуализации активности по часам и дням недели. Это позволяет найти «горячие часы» (например, «Среда, 19:00»), чтобы запланировать технические работы на время простоя, а маркетинговые акции — на пик активности.

Уровень: Средний
Что тренируем: *Двойную группировку (по дню и часу), извлечение компонентов времени (EXTRACT(HOUR)).*

Технический разбор

1. **Извлечение атрибутов (EXTRACT):** Получаем день недели и час дня (hour_of_day) из временной метки.
2. **Двойная группировка (GROUP BY):** Группируем данные сразу по двум измерениям (День + Час), создавая сетку для тепловой карты.
3. **Агрегация (SUM):** Суммируем выручку или активность для каждой ячейки этой сетки.
4. **Сортировка (ORDER BY):** Упорядочиваем результат, чтобы сразу увидеть самые активные временные слоты.

Совет

Часовые пояса - это главный враг тепловых карт. Данные в базе хранятся в UTC. Если ваши пользователи в Нью-Йорке, пик активности в "18:00 UTC" на самом деле означает "14:00 Local". Всегда приводите время к локальному таймзону основной аудитории (DATETIME(DateUTC, "Europe/Moscow")), иначе вы отправите пуш-уведомление, когда пользователи спят.
SQL выдает данные списком. Чтобы быстро задетектить паттерн, скопируйте результат в Excel или Google Sheets, сделайте сводную таблицу (Pivot Table: строки — Час, столбцы — День недели) и примените Условное форматирование (Color Scales).

Тепловая карта

Логика запроса

Мы нарезаем историю активности на сетку из двух измерений: день недели и час суток (например, «Понедельник, 14:00»). Затем мы подсчитываем количество операций в каждой ячейке этой сетки, чтобы найти самые «горячие» часы недели.

Пример кода

```sql
SELECT
  -- агрегация по дню недели и часу
  FORMAT_DATE('%A', DateUTC) as day_of_week,
  EXTRACT(HOUR FROM DateUTC) as hour_of_day_utc,
  -- подсчет транзакций и ревенью
  COUNT(PaymentAttemptID) as transactions_count,
  ROUND(SUM(RevenueUSD), 0) as total_revenue
FROM `project.dataset.payments`
GROUP BY day_of_week, hour_of_day_utc
ORDER BY total_revenue DESC
LIMIT 10; -- Смотрим топ-10 "горячих" часов
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details | Ex

Row	day_of_week	hour_of_day_utc	transactions_count	total_revenue
1	Saturday	0	56	2628.0
2	Monday	0	59	2405.0
3	Sunday	0	65	2308.0
4	Saturday	15	28	2212.0
5	Wednesday	0	57	2125.0
6	Saturday	22	40	2078.0
7	Saturday	1	55	2069.0
8	Sunday	7	41	2054.0
9	Sunday	1	52	2014.0
10	Friday	2	56	1998.0

Глава 4. Экономика и монетизация

В этой главе мы углубимся в экономику и посмотрим, как продукт зарабатывает деньги. Мы научимся препарировать выручку, чтобы понять не только её объем, но и её природу. Это критически важно для оценки устойчивости бизнеса: растем ли мы за счет агрессивного привлечения новичков или благодаря лояльности «старичков», и на ком на самом деле держится наша экономика.

Что вас ждет внутри главы

Анализ структуры выручки (New vs Recurring), расчет эффективности монетизации через ARPU и ARPPU, поиск оптимальных ценовых тиров (Price Points) и выявление «китов» — сверхплатящих пользователей, которые определяют финансовый успех продукта.

Зачем это в работе

Чтобы понимать качество роста. Разделение New/Recurring покажет, не превратился ли продукт в «сито», которое пропускает через себя трафик, не удерживая его. Анализ ценников поможет вычистить неэффективные офферы, а поиск «китов» позволит оценить риски: насколько сильно ваша экономика зависит от узкой группы VIP-клиентов.

Что осталось за кадром

Расходная часть и юнит-экономика. Мы фокусируемся на доходах (Revenue), но пока не учитываем затраты на маркетинг (CAC), налоги и комиссии сторов (это было в Главе 2) или стоимость содержания инфраструктуры.

Что почитать самостоятельно:

- Юнит-экономика (LTV > CAC / Payback Period / Contribution Margin)
- Закон Парето в данных (Concentration Analysis / Lorenz Curve)
- Модели монетизации (Subscription vs Transactional / MRR / Expansion Revenue)
- Психология и эластичность цен (Price Elasticity / Charm Pricing / Decoy Effect)

ARPU vs. ARPPU

Задача: Оценить финансовую отдачу от аудитории. Нам нужно рассчитать и сравнить два показателя: сколько денег приносит средний посетитель (включая тех, кто ничего не купил) и сколько тратит реальный покупатель. Это покажет, где у продукта «узкое горлышко»: в качестве трафика или в монетизации.

Уровень: Начинающий
Что тренируем: *CTE для подготовки данных, объединение несвязанных метрик (CROSS JOIN), арифметику.*

Технический разбор

1. **Сбор аудитории (CTE UserStats):** Считаем уникальных пользователей (total_users) в таблице регистраций.
2. **Сбор денег (CTE PayStats):** Считаем общую выручку (total_revenue) и уникальных платящих (paying_users) в таблице платежей.
3. **Объединение (CROSS JOIN):** Так как обе таблицы возвращают всего по одной строке (итоговые суммы), мы используем кросс-джойн, чтобы получить одну строку со всеми тремя числами.
4. **Финальный расчет (SELECT):** Делим полученные колонки друг на друга, используя NULLIF для защиты от деления на ноль.

Совет

ARPU (Average Revenue Per User) — доход делим на всех активных. Это ваш «потолок» стоимости закупки трафика (CPI). Если реклама стоит $2.00, а ARPU $1.50 — вы масштабируете убытки.
ARPPU (Average Revenue Per Paying User) — доход делим только на платящих. Эта метрика помогает тестировать цены. Например, вы подняли цену подписки: конверсия упала, но ARPPU вырос настолько, что перекрыл отток. Значит, решение верное.
ARPPU всегда будет больше ARPU. Если они равны — у вас 100% конверсия или ошибка в данных.

ARPU vs. ARPPU

Логика запроса

Мы готовим две независимые цифры. В первой «корзине» (таблица пользователей) считаем общее количество людей. Во второй «корзине» (таблица платежей) считаем общую выручку и количество платящих. Затем мы ставим эти корзины рядом и делим деньги на количество людей: сначала на всех (ARPU), потом только на платящих (ARPPU).

Пример кода

```sql
WITH UserStats AS (
  -- Считаем всех пользователей
  SELECT COUNT(DISTINCT UserID) as total_users
  FROM `project.dataset.users`
),
PayStats AS (
  -- Считаем деньги и платящих
  SELECT
    SUM(RevenueUSD) as total_revenue,
    COUNT(DISTINCT UserID) as paying_users
  FROM `project.dataset.payments`
)
SELECT
  p.total_revenue,
  u.total_users,
  p.paying_users,
  -- ARPU: Выручка на всех
  ROUND(p.total_revenue / NULLIF(u.total_users, 0), 2) as ARPU,
  -- ARPPU: Выручка на платящих
  ROUND(p.total_revenue / NULLIF(p.paying_users, 0), 2) as ARPPU
FROM UserStats u
CROSS JOIN PayStats p;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	total_revenue	total_users	paying_users	ARPU	ARPPU
1	181783.0300000...	59220	3261	3.07	55.74

Структура выручки: New vs Recurring

Задача: Понять природу роста. За счет чего растет выручка: мы привлекаем много новичков (New) или хорошо удерживаем старых (Recurring)? Здоровый SaaS или подписочный бизнес должен со временем расти именно за счет продлений.

Уровень: *Средний*
Что тренируем: *Группировку по атрибутам (GROUP BY), условное суммирование (CASE WHEN) внутри агрегатных функций.*

Технический разбор

1. **Округление времени (DATE_TRUNC):** Приводим все даты платежей к началу месяца для построения динамики.
2. **Сегментация (CASE WHEN):** Внутри функции SUM проверяем тип платежа. Если RenewalType = 'New', деньги идут в копилку «Новые», иначе — в «Продления».
3. **Расчет доли:** Делим сумму продлений на общую сумму, чтобы получить Stability Index (индекс устойчивости).
4. **Сортировка (ORDER BY):** Выводим месяцы от новых к старым.

Совет

В начале жизни продукта 100% выручки — это New. Но к концу первого года доля Recurring должна перевалить за 50% (для зрелых SaaS норма 70–80%). Важно понимать экономику: выручка New обычно уходит на покрытие расходов по привлечению (CAC), а Recurring — это ваша чистая прибыль. Для визуализации используйте график с накоплением (Stacked Area Chart). Здоровый бизнес выглядит как «слоеный пирог», где нижний слой (Recurring) постоянно утолщается, даже если верхний слой (New) остается стабильным. Если при отключении рекламы ваша выручка падает на 90% — у вас нет продукта, есть только арбитраж трафика.

Структура выручки: New vs Recurring

Логика запроса

Мы разделяем все платежи на два потока. Если транзакция помечена как New — это новый платеж. Если Auto-Renewal или Relist — это повторяющийся. Мы суммируем выручку по каждому типу помесячно и смотрим, какую долю занимают продления в общем котле.

Пример кода

```sql
SELECT
  DATE_TRUNC(DATE(DateUTC), MONTH) as month,
  -- Выручка с первых покупок
  ROUND(SUM(CASE WHEN RenewalType = 'New' THEN RevenueUSD ELSE 0 END), 2) as new_revenue,
  -- Выручка с продлений
  ROUND(SUM(CASE WHEN RenewalType != 'New' THEN RevenueUSD ELSE 0 END), 2) as recurring_revenue,
  -- Общая выручка
  ROUND(SUM(RevenueUSD), 2) as total_revenue,
  -- Доля продлений (Stability Index)
  ROUND(
    SUM(CASE WHEN RenewalType != 'New' THEN RevenueUSD ELSE 0 END) / SUM(RevenueUSD) * 100,
  1) as recurring_share_pct
FROM `project.dataset.payments`
GROUP BY month
ORDER BY month DESC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	month	new_revenue	recurring_revenue	total_revenue	recurring_share...
1	2025-08-01	23805.97	15687.72	39493.69	39.7
2	2025-07-01	21423.56	18719.76	40143.32	46.6
3	2025-06-01	22689.28	13040.72	35730.0	36.5
4	2025-05-01	22394.95	11705.97	34100.92	34.3
5	2025-04-01	20210.98	11938.36	32149.34	37.1
6	2025-03-01	135.82	29.94	165.76	18.1

Продлеваемость подписок

Задача: Измерить лояльность рублем. Метрика показывает, какой процент пользователей совершил повторную оплату после первой покупки. Это более "честная" метрика удержания, чем просто посещаемость.

Уровень: Продвинутый
Что тренируем: *Оконные функции навигации (LEAD), работу с подзапросами (CTE), логику когорт.*

Технический разбор

1. **Подготовка цепочки (CTE PaymentChain):** Для каждого платежа с помощью функции LEAD находим дату следующего платежа этого же пользователя.
2. **Фильтрация базы (RenewalType = 'New'):** Нас интересует судьба только новых пользователей, поэтому во внешнем запросе оставляем только RenewalType = 'New'.
3. **Флаг продления:** Если колонка next_payment_date не пустая (IS NOT NULL), значит, продление состоялось.
4. **Агрегация:** Считаем процент успешных продлений по месяцам.

Совет

Самый критичный момент — переход с 1-го на 2-й месяц (Renewal Rate обычно падает до 40–60%). Всегда разделяйте отток на активный (юзер сам отменил подписку в настройках) и пассивный (ошибка оплаты, истекла карта, недостаточно средств). Часто до 30% всех непродлений — это именно технические сбои (Involuntary Churn). Прежде чем менять продукт или цены, проверьте настройки dunning (логику повторных попыток списания) в вашем платежном шлюзе — это самый простой и дешевый способ вернуть деньги.

Продлеваемость подписок

Логика запроса

Мы выстраиваем платежи каждого пользователя в хронологическую цепочку. Для каждой покупки мы "подглядываем" вперед: есть ли у этого же человека следующая запись с более поздней датой? Если есть — ставим "зачет". В конце делим количество тех, кто продлился, на общее число новичков.

Пример кода

```sql
WITH PaymentChain AS (
  SELECT
    UserID,
    DateUTC,
    RenewalType,
    -- Смотрим, когда был следующий платеж у этого же юзера
    LEAD(DateUTC) OVER (PARTITION BY UserID ORDER BY DateUTC) as next_payment_date
  FROM `ga-local-data-extract.interview_data.payments`
)
SELECT
  DATE_TRUNC(DATE(DateUTC), MONTH) as cohort_month,
  -- База: только первые покупки
  COUNT(UserID) as new_subscribers,
  -- Сколько из них имели следующий платеж
  COUNT(CASE WHEN next_payment_date IS NOT NULL THEN UserID END) as renewed_users,
  -- Процент продления
  ROUND(
    COUNT(CASE WHEN next_payment_date IS NOT NULL THEN UserID END) / COUNT(UserID) * 100,
  1) as renewal_rate_pct
FROM PaymentChain
WHERE RenewalType = 'New' -- Анализируем судьбу только новичков
GROUP BY cohort_month
ORDER BY cohort_month DESC;
```

Ожидаемый результат

Row	cohort_month	new_subscribers	renewed_users	renewal_rate_pct
1	2025-08-01	638	66	10.3
2	2025-07-01	599	102	17.0
3	2025-06-01	549	129	23.5
4	2025-05-01	619	178	28.8
5	2025-04-01	534	153	28.7
6	2025-03-01	6	1	16.7

Анализ ценовых точек

Задача: Найти реальные ценовые тиры. Из-за налогов и комиссий суммы платежей в базе «плавают» (например, $28.99 и $29.15). Нам нужно очистить этот шум и понять, какие базовые тарифы приносят основную выручку, чтобы упростить продуктовую линейку.

Уровень: Начинающий
Что тренируем: *округление (ROUND), подзапросы (CTE), объединение итогов (CROSS JOIN).*

Технический разбор

1. **Сбор общего итога (CTE TotalStats):** Находим суммарную выручку (grand_total) по всем транзакциям без фильтров.
2. **Округление и группировка (GROUP BY):** Применяем ROUND(RevenueUSD, 0), чтобы превратить «шумные» ценники (28.83, 29.18) в единую точку «29».
3. **Объединение (CROSS JOIN):** Приклеиваем общую сумму к каждой строке сгруппированного списка.
4. **Расчет доли (SELECT):** Вычисляем процентный вклад каждого ценового кластера в общую выручку.

Совет

Если низкие чеки составляют 80% транзакций, но дают меньше 10% выручки — это повод для агрессивного эксперимента. Попробуйте отключить их: часть юзеров уйдет, но «ядро» перетечет в более дорогие тарифы, что поднимет маржинальность без роста нагрузки на поддержку.
Для визуализации можно использовать диаграмму Парето (Pareto Chart). Это наглядно покажет какой процент ценовых точек дает соответствующий процент выручки.

Анализ ценовых точек

Логика запроса

Мы создаем две корзины. В первой — общая выручка всего проекта. Во второй — группируем платежи, предварительно округляя сумму каждого чека до целого доллара. Это позволяет «склеить» разрозненные транзакции в единые ценовые категории. Затем мы делим доход каждой категории на общий итог.

Пример кода

```sql
WITH TotalStats AS (
  -- Шаг 1: Считаем общую выручку для знаменателя
  SELECT SUM(RevenueUSD) as grand_total
  FROM `project.dataset.payments`
)
SELECT
  ROUND(p.RevenueUSD, 0) as price_tier, -- Округляем для борьбы с "шумом"
  COUNT(p.PaymentAttemptID) as purchases_count,
  ROUND(SUM(p.RevenueUSD), 0) as total_revenue,
  -- Шаг 2: Считаем долю кластера в общем доходе
  ROUND(
    SUM(p.RevenueUSD) / MAX(t.grand_total) * 100,
    1
  ) as revenue_share_pct
FROM `project.dataset.payments` p
CROSS JOIN TotalStats t
GROUP BY price_tier -- Группируем по округленной цене
ORDER BY total_revenue DESC
LIMIT 15; -- Берем ТОП самых значимых ценников
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details

Row	price_tier	purchases_count	total_revenue	revenue_share_pct
1	29.0	1195	34689.0	19.1
2	33.0	491	16177.0	8.9
3	58.0	194	11262.0	6.2
4	52.0	208	10838.0	6.0
5	22.0	320	7054.0	3.9
6	34.0	141	4749.0	2.6
7	25.0	188	4739.0	2.6

Поиск «Китов»

Задача: Проверить продукт на устойчивость через закон Парето. Нам нужно понять, какая доля выручки приходится на топ-10% самых платящих пользователей («Китов»). Если бизнес держится на узкой группе сверхплатящих фанатов, это создает риск высокой зависимости от лояльности крошечного сегмента аудитории.

Уровень: Продвинутый
Что тренируем: *функции ранжирования (NTILE), оконные функции для расчета долей (OVER), агрегацию данных по группам.*

Технический разбор

1. **Расчет LTV (CTE UserLTV):** Группируем таблицу платежей по UserID и находим общую сумму RevenueUSD для каждого уникального покупателя.
2. **Разбиение на децили (CTE Deciles):** Используем функцию NTILE(10) над рассчитанным LTV. Она присваивает каждому юзеру номер группы от 1 (максимальные чеки) до 10.
3. **Агрегация по группам (SELECT):** Группируем данные по номеру дециля, подсчитывая количество людей и сумму денег в каждой группе.
4. **Расчет доли (Window Function):** Используем конструкцию SUM(SUM(...)) OVER() для получения глобального знаменателя и вычисления процентного вклада каждого дециля.

Совет

Если низкие чеки составляют 80% транзакций, но дают меньше 10% выручки — это повод для агрессивного эксперимента. Попробуйте отключить их: часть юзеров уйдет, но «ядро» перетечет в более дорогие тарифы, что поднимет маржинальность без роста нагрузки на поддержку.
Для визуализации можно использовать диаграмму Парето (Pareto Chart). Это наглядно покажет какой процент ценовых точек дает соответствующий процент выручки.

Поиск «Китов»

Логика запроса

Мы выстраиваем всех покупателей в очередь по сумме их трат (LTV) и нарезаем эту очередь на десять равных частей — децилей. В первую корзину попадает топ-10% самых щедрых клиентов, в последнюю — самые экономные. Затем мы подсчитываем суммарный доход внутри каждой корзины и вычисляем его процентный вклад в общий объем выручки.

Пример кода

```sql
WITH UserLTV AS (
  -- Шаг 1: Находим общую ценность каждого платящего клиента
  SELECT
    UserID,
    SUM(RevenueUSD) as total_ltv
  FROM `project.dataset.payments`
  GROUP BY UserID
),
RankedUsers AS (
  -- Шаг 2: Нарезаем плательщиков на 10 равных групп по убыванию LTV
  SELECT
    UserID,
    total_ltv,
    NTILE(10) OVER (ORDER BY total_ltv DESC) as decile
  FROM UserLTV
)
SELECT
  decile,
  COUNT(UserID) as users_count,
  ROUND(SUM(total_ltv), 0) as decile_revenue,
  -- Шаг 3: Считаем вклад группы в % от общего "пирога"
  ROUND(
    SUM(total_ltv) / SUM(SUM(total_ltv)) OVER() * 100,
    1
  ) as revenue_share_pct
FROM RankedUsers
GROUP BY decile
ORDER BY decile ASC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details

Row	decile	users_count	decile_revenue	revenue_share_pct
1	1	327	65035.0	35.8
2	2	326	26668.0	14.7
3	3	326	18885.0	10.4

Глава 5. Сегментация

В этой главе мы осваиваем один из самых мощных и универсальных инструментов аналитика. Мы разбираем сегментация по семейному положению или количеству фото — но это лишь примеры. Те же самые подходы можно и нужно применять для сегментации по платформам (iOS/Android), географии, типам устройств или любым другим доступным признакам. Наша задача — научиться разделять генеральную совокупность на четкие группы, поведение которых можно сравнивать.

Что вас ждет внутри главы

Методы работы с разными типами данных для создания сегментов: сравнение групп на основе текстовых полей (категорийная сегментация), трансформация числовых показателей в логические интервалы (поведенческие бакеты) и построение комплексной системы оценки лояльности через RFM-скоринг.

Зачем это в работе

Чтобы понимать, «где лежат деньги». Сегментация позволяет увидеть, что за средними показателями скрываются абсолютно разные паттерны: например, одна группа пользователей может приносить 80% выручки, а другая — лишь генерировать нагрузку на поддержку. Это помогает продукту сфокусироваться на «золотых когортах» и находить «Ага-моменты» — конкретные действия (например, загрузка 3-го фото), после которых вовлеченность пользователя резко возрастает.

Что осталось за кадром

Автоматическая кластеризация. Мы учимся сегментировать базу на основе экспертных правил и бизнес-логики. За кадром остаются алгоритмы машинного обучения (например, K-means), которые ищут скрытые связи в данных без участия человека.

Что почитать самостоятельно:

- Парадокс Симпсона (Aggregated vs Segmented Data)
- RFM-анализ (Recency, Frequency, Monetary)
- ABC/XYZ-анализ (Revenue Contribution / Sales Predictability)

Категорийная сегментация

Задача: Выявить наиболее прибыльные группы пользователей по доступной фиче - семейному положению. Это поможет точнее настраивать таргетинг в рекламных кабинетах и адаптировать маркетинговые сообщения под конкретные жизненные сценарии клиентов.

Уровень: Начинающий
Что тренируем: *соединение таблиц (LEFT JOIN), группировку по строковому полю (GROUP BY), использование NULLIF для безопасных расчетов.*

Технический разбор

1. **Объединение (LEFT JOIN):** Соединяем основную таблицу users с таблицей payments по ключу UserID, чтобы не потерять тех, кто еще не совершал покупок.
2. **Группировка (GROUP BY):** Схлопываем данные по текстовому полю MaritalStatus.
3. **Агрегация (SELECT):** Считаем общее количество регистраций в сегменте, число уникальных платящих пользователей и суммарный доход.
4. **Расчет ARPPU:** Делим выручку на количество платящих пользователей внутри каждой категории, используя NULLIF для исключения ошибок при делении на ноль.

Совет

Сегментация — это единственный способ увидеть реальную структуру бизнеса. Группировка по MaritalStatus — лишь пример. Тот же самый SQL-запрос позволит вам сегментировать базу по типу устройства (iOS/Android), версии приложения или источнику трафика. Ваша цель как аналитика — найти «выбросы»: группы, где ARPPU в 2–3 раза выше среднего. Обнаружив прибыльный сегмент, не торопитесь праздновать. Проверьте объем этой группы (total_users). Для бизнеса важны те группы, которые сочетают в себе и высокую доходность, и значительный масштаб.

Категорийная сегментация

Логика запроса

Мы берем всех пользователей и «приклеиваем» к ним историю платежей. Затем мы распределяем данные по корзинам в соответствии с семейным положением. В каждой корзине мы считаем три показателя: общий объем аудитории, количество реальных покупателей и сумму принесенных ими денег. В финале мы делим выручку на количество платящих, чтобы сравнить ценность одного клиента (ARPPU) в каждом сегменте.

Пример кода

```sql
SELECT
  -- Сегментом может быть любая категория: статус, платформа или страна
  u.MaritalStatus as segment,
  COUNT(DISTINCT u.UserID) as total_users,
  COUNT(DISTINCT p.UserID) as paying_users,
  ROUND(SUM(p.RevenueUSD), 0) as total_revenue,
  -- ARPPU: Средний чек на одного платящего внутри сегмента
  ROUND(
    SUM(p.RevenueUSD) / NULLIF(COUNT(DISTINCT p.UserID), 0),
    2
  ) as arppu
-- Используем DISTINCT, чтобы дубли в users не размножали строки с платежами
FROM (SELECT DISTINCT UserID, MaritalStatus FROM `project.dataset.users`) u
LEFT JOIN `project.dataset.payments` p
  ON u.UserID = p.UserID
GROUP BY 1
ORDER BY total_revenue DESC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	segment	total_users	paying_users	total_revenue	arppu
1	Single	24839	1675	90318.0	53.92
2	(unknown)	22294	531	28747.0	54.14
3	Divorced	2610	413	26385.0	63.89
4	Separated	2623	265	13681.0	51.63
5	Widowed	2489	163	8818.0	54.1
6	Prefer Not Say	2793	137	8371.0	61.1
7	Married	352	44	3640.0	82.73
8	Other	1220	33	1823.0	55.23

Поведенческая сегментация

Задача: Найти «точку перелома» в вовлеченности. Нам нужно понять, как количество загруженных фотографий влияет на готовность пользователя платить. Это позволит определить идеальный порог заполнения профиля, при котором конверсия в покупку становится максимальной.

Уровень: Средний
Что тренируем: *создание категорий из чисел (CASE WHEN), левое соединение (LEFT JOIN), расчет относительных метрик (CR, ARPU).*

Технический разбор

1. **Категоризация (CASE WHEN):** Группируем сырое число PhotoCount в логические интервалы. Это превращает тысячи уникальных значений в 4–5 понятных сегментов.
2. **Объединение (LEFT JOIN):** Привязываем платежи к пользователям, чтобы сопоставить их активность с финансовым результатом.
3. **Группировка (GROUP BY):** Схлопываем данные по созданным интервалам (бакетам).
4. **Расчет эффективности (SELECT):** Вычисляем конверсию в плательщика (сколько % из группы совершили покупку) и ARPU (сколько денег в среднем принес один пользователь из этой группы).

Совет

Поведенческая сегментация — лучший способ найти «Ага-момент» вашего продукта. Если данные показывают, что конверсия в покупку резко вырастает именно после загрузки третьего фото, значит, ваша главная цель в онбординге — заставить пользователя загрузить именно три фотографии. Любые пуши и подсказки должны «бить» в эту точку. Не ограничивайтесь только деньгами. Посмотрите, как бакеты с анализируемым сегментом коррелируют с удержанием (Retention) или другими важными метриками.

Поведенческая сегментация

Логика запроса

Мы нарезаем аудиторию на «корзины» (бакеты) по количеству фото: например, 0, 1–2, 3–5 и более 5. Затем для каждой корзины мы считаем общее количество людей, число платящих и итоговую выручку. В финале мы сравниваем конверсию (CR) и средний доход на пользователя (ARPU) между группами, чтобы увидеть, как контент влияет на деньги.

Пример кода

```sql
WITH UserBuckets AS (
  -- Шаг 1: Нарезаем пользователей на поведенческие группы (с защитой от дублей)
  SELECT DISTINCT
    UserID,
    CASE
      WHEN PhotoCount = 0 THEN '0: No Photos'
      WHEN PhotoCount BETWEEN 1 AND 2 THEN '1: Low (1-2)'
      WHEN PhotoCount BETWEEN 3 AND 5 THEN '2: Medium (3-5)'
      ELSE '3: High (6+)'
    END as photo_bucket
  FROM `project.dataset.users`
)
SELECT
  b.photo_bucket,
  COUNT(DISTINCT b.UserID) as total_users,
  COUNT(DISTINCT p.UserID) as paying_users,
  -- Конверсия в покупателя (Conversion Rate)
  ROUND(COUNT(DISTINCT p.UserID) / COUNT(DISTINCT b.UserID) * 100, 2) as cr_percent,
  -- Средний доход на пользователя (ARPU)
  ROUND(SUM(p.RevenueUSD) / COUNT(DISTINCT b.UserID), 2) as arpu
FROM UserBuckets b
LEFT JOIN `project.dataset.payments` p
  ON b.UserID = p.UserID
GROUP BY photo_bucket
ORDER BY photo_bucket ASC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	photo_bucket	total_users	paying_users	cr_percent	arpu
1	0: No Photos	19391	424	2.19	1.2
2	1: Low (1-2)	29802	1583	5.31	2.84
3	2: Medium (3-5)	10027	1254	12.51	7.39

RFM-скоринг

Задача: Разделить платящую базу на сегменты по их «жизненным показателям»: давности, частоте и сумме покупок. Это позволяет перестать рассылать одинаковые предложения всей базе и начать работать точечно: возвращать уходящих, поощрять лояльных и стимулировать новичков к повторной оплате.

Уровень: Продвинутый
Что тренируем: *функции ранжирования (NTILE), агрегацию данных, объединение строк (CONCAT).*

Технический разбор

1. **Агрегация (CTE UserMetrics):** Для каждого UserID находим дату самого свежего платежа, общее число транзакций и сумму всех оплат.
2. **Ранжирование (CTE RFM_Scores):** С помощью NTILE(3) разбиваем пользователей на три равные корзины по каждому параметру. Важно: для даты оплаты (R) более поздняя дата получает более высокий балл.
3. **Сборка индекса (SELECT):** Склеиваем три цифры в одну строку (RFM-код). Это превращает набор метрик в удобный идентификатор сегмента.
4. **Анализ сегментов:** Группируем базу по RFM-коду, чтобы увидеть размер каждой группы и её ценность.

Совет

Если ваш ключевой сегмент «333» (покупают часто, много и недавно) начинает редеть, а сегмент «133» (платили часто и много, но давно) растет — у продукта критические проблемы с удержанием. Это сигнал к действию. Не пытайтесь работать со всеми комбинациями кодов. Сфокусируйтесь на трех: «Чемпионы» (333) — дайте им статус VIP; «Новички» (311/312) — предложите бонус за вторую покупку; «Уходящие киты» (133) — отправьте персональное пуш-уведомление.

RFM-скоринг

Логика запроса

Мы собираем платежный профиль каждого клиента в три показателя: дату последней оплаты (Recency), общее количество транзакций (Frequency) и суммарную выручку (Monetary). Затем мы делим базу на три равные группы по каждому параметру, присваивая баллы от 1 (слабо) до 3 (сильно). Объединив эти баллы в единый код (например, «333»), мы получаем наглядную сегментацию от «чемпионов» до «потерянных».

Пример кода

```sql
WITH UserMetrics AS (
  -- Шаг 1: Агрегируем поведение каждого платящего юзера
  SELECT
    UserID,
    DATE(MAX(DateUTC)) as last_pay_date,
    COUNT(PaymentAttemptID) as freq,
    SUM(RevenueUSD) as mon
  FROM `project.dataset.payments`
  GROUP BY UserID
),
RFM_Scores AS (
  -- Шаг 2: Присваиваем баллы от 1 до 3 (3 – лучший результат)
  SELECT
    UserID,
    mon,
    NTILE(3) OVER (ORDER BY last_pay_date ASC) as r,
    NTILE(3) OVER (ORDER BY freq ASC) as f,
    NTILE(3) OVER (ORDER BY mon ASC) as m
  FROM UserMetrics
)
SELECT
  -- Шаг 3: Формируем итоговый сегмент
  CONCAT(CAST(r AS STRING), CAST(f AS STRING), CAST(m AS STRING)) as rfm_segment,
  COUNT(UserID) as users_count,
  ROUND(AVG(mon), 2) as avg_monetary
FROM RFM_Scores
GROUP BY 1
ORDER BY rfm_segment DESC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Executi

Row	rfm_segment	users_count	avg_monetary
1	333	389	126.24
2	332	170	37.2
3	331	150	23.02

Глава 6. A/B Тестирование

В этой главе мы переходим к ответственному этапу — проверке гипотез. Здесь мы делаем важное допущение:мы коснемся этой темы по касательной. Мы используем SQL как инструмент для подготовки данных для дальнейшего анализа. Мы не будем закапываться в математические дебри - про это можно написать отдельную книгу. Наша задача — научить вас быстро и чисто выгружать ключевые агрегаты, которые потом можно «скормить» любому статистическому калькулятору.

Что вас ждет внутри главы

Методика расчета основного результата через Uplift конверсии, подготовка базы (среднее, дисперсия, размер выборки) для анализа среднего чека и проверка стабильности данных через кумулятивные метрики.

Зачем это в работе

Чтобы принимать решения на основе фактов, а не интуиции. Умение самостоятельно подготовить данные для теста — это признак автономности аналитика. Эти запросы позволяют быстро понять: действительно ли новая фича «перформит» или вы просто видите случайный шум.

Что осталось за кадром

Весь глубокий статистический аппарат. За кадром остаются расчеты p-value, оценка мощности теста (Power) и сложные дизайны вроде многоруких бандитов или последовательного анализа.

Что почитать самостоятельно:

- Статистическая значимость (P-value / Confidence Intervals / Significance Level)
- Статистические критерии (T-test / Mann-Whitney U-test / Bootstrap)
- Типы ошибок в тестах (Type I & Type II Errors / False Positive vs False Negative)
- Проблемы чистоты эксперимента (SRM — Sample Ratio Mismatch / Peeking Problem / Network Effects)

Конверсия в покупку и расчет Uplift

Задача: Оценить эффективность изменений в продукте. Нам нужно сравнить конверсию в покупку в тестовой и контрольной группах и рассчитать Uplift — относительный прирост, который покажет, насколько лучше (или хуже) сработала новая версия продукта.

Уровень: Средний
Что тренируем: *оконные функции для обращения к значениям других строк (OVER), условную агрегацию (CASE WHEN), расчет относительных изменений.*

Технический разбор

1. **Сбор базы (CTE BaseStats):** Соединяем пользователей с платежами и группируем по полю ControlOrTest, подсчитывая общее количество «голов» и уникальных плательщиков.
2. **Расчет чистой конверсии (CTE CalcCR):** Делим плательщиков на общее количество пользователей в каждой группе. Используем NULLIF, чтобы избежать ошибки при делении на ноль.
3. **Выделение эталона (MAX + CASE):** В основном запросе используем оконную функцию, чтобы «вытащить» значение конверсии контрольной группы в отдельную виртуальную колонку для сравнения.
4. **Расчет Uplift:** Применяем формулу (Test_CR / Control_CR - 1) * 100, чтобы получить процентный вклад эксперимента.

Совет

Не спешите с выводами, если Uplift составил +20%, но в группах всего по паре сотен человек. При малых выборках такой результат может быть игрой случая. Всегда проверяйте статзначимость (p-value) через внешние калькуляторы, прежде чем раскатывать фичу на всех. Если новая фича уронила конверсию, проведите сегментацию. Обновление могло «зайти» активным пользователям, но отпугнуло новичков. Всегда проверяйте корректность сплитования (разделения на группы).

Конверсия в покупку и расчет Uplift

Логика запроса

Мы создаем две корзины на основе признака группы эксперимента. В каждой корзине мы считаем общее количество пользователей и количество тех, кто совершил хотя бы одну покупку. Затем мы вычисляем процент конверсии (CR) для каждой группы. В финале мы сравниваем показатель теста с показателем контроля, чтобы выразить разницу в процентах прироста.

Пример кода

```sql
WITH BaseStats AS (
  -- Шаг 1: Считаем охват и количество покупателей по группам
  SELECT
    u.ControlOrTest,
    COUNT(DISTINCT u.UserID) as users,
    COUNT(DISTINCT p.UserID) as payers
  FROM `project.dataset.users` u
  LEFT JOIN `project.dataset.payments` p
    ON u.UserID = p.UserID
  GROUP BY 1
),
CalcCR AS (
  -- Шаг 2: Вычисляем базовую конверсию (CR)
  SELECT
    ControlOrTest,
    users,
    payers,
    payers / NULLIF(users, 0) as cr
  FROM BaseStats
)
SELECT
  ControlOrTest as experiment_group,
  users,
  payers,
  ROUND(cr * 100, 2) as cr_percent,
  -- Шаг 3: Рассчитываем Uplift относительно группы Control
  ROUND(
    (cr / MAX(CASE WHEN ControlOrTest = 'Control' THEN cr END) OVER() - 1) * 100,1
    ) as uplift_percent
FROM CalcCR;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	experiment_group	users	payers	cr_percent	uplift_percent
1	Control	29728	1612	5.42	0.0
2	Test	29492	1649	5.59	3.1

Сравнение среднего чека

Задача: Проверить качество монетизации. Рост конверсии часто сопровождается падением среднего чека (например, если привлечено много «дешевых» пользователей). Нам нужно подготовить статистическую базу — среднее значение, стандартное отклонение и размер выборки, — чтобы понять, является ли разница в доходах между группами значимой.

Уровень: Средний
Что тренируем: *статистические агрегаты (AVG, STDDEV_SAMP), объединение таблиц по ключу (JOIN), фильтрацию.*

Технический разбор

1. **Расчет выручки (CTE UserChecks):** Группируем таблицу платежей по UserID и находим суммарный доход (user_ltv) для каждого уникального покупателя.
2. **Привязка групп (JOIN):** Соединяем таблицу users с результатами из CTE. Используем INNER JOIN, так как для анализа чека нам нужны только те, кто совершил покупку.
3. **Группировка (GROUP BY):** Распределяем данные по признаку ControlOrTest.
4. **Стат-агрегация (SELECT):** Вычисляем среднее арифметическое (AVG), размер выборки (COUNT) и стандартное отклонение (STDDEV_SAMP) для каждой группы.

Совет

Средний чек крайне чувствителен к «выбросам». Стандартное отклонение (std_dev) показывает этот разброс: чем оно выше, тем менее устойчиво ваше «среднее». Если отклонение в разы превышает средний чек, значит, результат теста определяют крупные транзакции, а не системное улучшение. Полученные цифры (N, Mean, StdDev) — это готовый входной набор для T-теста Стьюдента. SQL идеально подходит для подготовки этих агрегатов, но расчет p-value лучше проводить в специализированных калькуляторах или Python-скриптах.

Сравнение среднего чека

Логика запроса

Мы готовим две независимые корзины данных для платящих пользователей из контрольной и тестовой групп. Сначала мы находим итоговую сумму трат (LTV) для каждого человека. Затем, сгруппировав их по признаку эксперимента, рассчитываем три ключевых показателя: количество покупателей, их средний чек и степень разброса данных (стандартное отклонение).

Пример кода

```sql
WITH UserChecks AS (
  -- Шаг 1: Находим суммарный доход по каждому платящему клиенту
  SELECT
    UserID,
    SUM(RevenueUSD) as user_ltv
  FROM `project.dataset.payments`
  GROUP BY UserID
)
SELECT
  u.ControlOrTest as experiment_group,
  -- Шаг 2: Готовим математическую базу для оценки значимости
  COUNT(uc.UserID) as sample_size,
  ROUND(AVG(uc.user_ltv), 2) as mean_revenue,
  ROUND(STDDEV_SAMP(uc.user_ltv), 2) as std_dev
-- Дедупликация юзеров необходима для корректного расчета AVG и STDDEV
FROM (SELECT DISTINCT UserID, ControlOrTest FROM `project.dataset.users`) u
INNER JOIN UserChecks uc ON u.UserID = uc.UserID
GROUP BY u.ControlOrTest;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details

Row	experiment_group	sample_size	mean_revenue	std_dev
1	Control	1612	58.94	110.49
2	Test	1649	52.62	56.17

Кумулятивная выручка по группам

Задача: Проверить сходимость A/B теста. Нужно понять, стабилизировались ли результаты эксперимента или они всё еще подвержены сильным колебаниям. Это помогает принять решение об остановке теста, когда кумулятивные графики перестают пересекаться и выходят на «плато».

Уровень: Средний
Что тренируем: *оконные функции для накопительного итога (SUM() OVER), предварительную агрегацию в CTE, хронологическую сортировку.*

Технический разбор

1. **Агрегация по дням (CTE DailyRevenue):** Группируем платежи по дате и группе эксперимента, вычисляя сумму дохода, полученного за конкретные сутки.
2. **Определение окна (OVER):** В основном запросе задаем окно, которое группируется по признаку ControlOrTest и упорядочивается по дате.
3. **Накопительный итог (ROWS BETWEEN):** Инструкция UNBOUNDED PRECEDING AND CURRENT ROW заставляет SQL суммировать значения от самой первой записи до текущего дня включительно.
4. **Хронологическая сортировка:** Упорядочиваем финальный результат, чтобы данные были готовы для построения линейного графика.

Совет

Кумулятивные графики отлично подсвечивают проблему «отравления» теста: если в одну из групп случайно попал «кит» (сверхплатящий юзер), на линии будет виден резкий вертикальный скачок (ступенька). Не останавливайте тест, пока линии на графике кумулятивной выручки пересекаются или идут под острым углом друг к другу. Дождитесь параллельности — это признак того, что данные «сошлись» и случайные выбросы (например, пара случайных крупных покупок) больше не влияют на общую картину.

Кумулятивная выручка по группам

Логика запроса

Мы нарезаем историю теста на временные отрезки (дни). Для каждого дня в обеих группах (Control/Test) мы сначала считаем выручку, полученную именно в этот день. Затем мы используем «накопительный итог», прибавляя к выручке текущего дня всё, что было заработано ранее. Это сглаживает ежедневные скачки и показывает общую динамику наполнения бюджета в каждой группе.

Пример кода

```sql
WITH DailyRevenue AS (
  -- Шаг 1: Считаем выручку за каждый день для каждой группы
  SELECT
    DATE(p.DateUTC) as pay_date,
    u.ControlOrTest,
    SUM(p.RevenueUSD) as day_revenue
  -- Защита: схлопываем юзеров, чтобы не раздувать сумму платежей при джойне
  FROM (SELECT DISTINCT UserID, ControlOrTest FROM `project.dataset.users`) u
  JOIN `project.dataset.payments` p
    ON u.UserID = p.UserID
  GROUP BY 1, 2
)
SELECT
  pay_date,
  ControlOrTest,
  ROUND(day_revenue, 0) as daily_revenue,
  -- Шаг 2: Считаем сумму с начала теста до текущей даты
  ROUND(SUM(day_revenue) OVER (
    PARTITION BY ControlOrTest
    ORDER BY pay_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ), 0) as cumulative_revenue
FROM DailyRevenue
ORDER BY pay_date, ControlOrTest;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details

Row	pay_date	ControlOrTest	daily_revenue	cumulative_reven...
1	2025-03-31	Control	127.0	127.0
2	2025-03-31	Test	38.0	38.0
3	2025-04-01	Control	416.0	543.0
4	2025-04-01	Test	345.0	383.0

ГЛАВА 7. Жизненный цикл

В финальной главе мы переводим наши расчеты из календарной плоскости в систему координат «жизни пользователя». Мы отходим от анализа выручки за конкретный месяц и переходим к анализу когорт — групп людей, пришедших в продукт за определенный период времени. Наша задача — зафиксировать реальную историю их удержания и накопленную ценность (LTV). Мы не будем строить сложные прогнозы на будущее, а научимся чисто и точно считать то, что уже произошло в жизненном цикле клиента.

Что вас ждет внутри главы

Методики расчета классического удержания (Retention Day N) для оценки «липкости» продукта, замер скорости «утечки» платящей базы (Churn Rate) и построение модели накопительного LTV для оценки реальной окупаемости каждой когорты.

Зачем это в работе

Чтобы понять, жизнеспособен ли бизнес в долгосрочной перспективе. Когортный анализ LTV — это единственный честный способ ответить на вопрос: «Когда окупятся деньги, вложенные в рекламу?». Эти отчеты становятся главным аргументом при обсуждении развития и маркетинга продукта.

Что осталось за кадром

Сложное прогнозирование LTV на 2–3 года вперед и алгоритмы Churn Prediction. Мы учимся фиксировать реальность: сколько пользователь уже принес и как быстро он на самом деле уходит.

Что почитать самостоятельно:

- Когортный анализ (Time-based Cohorts / Retention Triangle / Heatmaps)
- Product-Market Fit (Retention Curve / Flatline Retention / Market Validation)
- Экономика удержания (LTV to CAC Ratio / Payback Period / Net Revenue Retention)
- Моделирование LTV (Historical vs Predictive LTV / Customer Journey Lifetime)

Retention Day N

Задача: Оценить «живучесть» продукта. Нам нужно понять, какой процент новичков возвращается и совершает целевое действие (в нашем случае — платеж) спустя 1, 7 или 30 дней после регистрации. Это главный индикатор того, нашли ли мы Product-Market Fit: если пользователи не возвращаются, маркетинг превращается в «наполнение дырявого ведра».

Уровень: Продвинутый
Что тренируем: *расчет разницы дат (DATE_DIFF), когортный анализ, условную агрегацию (CASE WHEN) для построения матриц.*

Технический разбор

1. **Когорты (CTE Cohorts):** Фиксируем дату регистрации (reg_date) для каждого пользователя. Это точка отсчета для нашего анализа.
2. **Активность (CTE Activity):** Соединяем таблицу когорт с платежами и вычисляем «возраст» каждой транзакции (day_n) как разницу в днях между покупкой и регистрацией.
3. **Когортная матрица (SELECT):** Группируем данные по дате регистрации. С помощью COUNT(DISTINCT CASE ...) мы подсчитываем, сколько уникальных людей из каждой когорты совершили действие в строго определенный день жизни.
4. **Расчет Retention:** Делим количество активных пользователей Day N на размер исходной когорты и переводим в проценты.

Совет

Day 1 Retention обычно отражает качество онбординга и то, насколько быстро пользователь понял ценность приложения. Day 7 показывает формирование привычки, а Day 30 — долгосрочную ценность. Если кривая удержания со временем не выходит на плато (горизонтальную линию), а падает в ноль — ваш продукт нежизнеспособен. Плато показывает наличие ядра лояльной аудитории, на котором строится экономика.

Retention Day N

Логика запроса

Мы фиксируем «день рождения» каждого пользователя (дату регистрации). Затем мы смотрим на все его последующие действия в таблице платежей и вычисляем разницу в днях между датой покупки и датой регистрации. Мы распределяем этих пользователей по временным корзинам: «активен в день 1», «в день 7» и т.д. В финале мы делим количество уникальных пользователей, вернувшихся в конкретный день, на общее число новичков в когорте.

Пример кода

```sql
WITH Cohorts AS (
  -- Шаг 1: Определяем дату регистрации для каждого пользователя
  SELECT
    UserID,
    DATE(DateUTC) as reg_date
  FROM `project.dataset.users`
),
Activity AS (
  -- Шаг 2: Находим "возраст" каждого действия пользователя в днях
  SELECT
    c.UserID,
    c.reg_date,
    DATE_DIFF(DATE(p.DateUTC), c.reg_date, DAY) as day_n
  FROM Cohorts c
  JOIN `project.dataset.payments` p
    ON c.UserID = p.UserID
)
SELECT
  reg_date,
  COUNT(DISTINCT UserID) as cohort_size,
  -- Шаг 3: Рассчитываем долю вернувшихся в ключевые дни жизни
  ROUND(COUNT(DISTINCT CASE WHEN day_n = 1 THEN UserID END) / NULLIF(COUNT(DISTINCT UserID), 0) * 100, 1) as day_1_retention_pct,
  ROUND(COUNT(DISTINCT CASE WHEN day_n = 7 THEN UserID END) / NULLIF(COUNT(DISTINCT UserID), 0) * 100, 1) as day_7_retention_pct,
  ROUND(COUNT(DISTINCT CASE WHEN day_n = 30 THEN UserID END) / NULLIF(COUNT(DISTINCT UserID), 0) * 100, 1) as day_30_retention_pct
FROM Activity
GROUP BY 1
ORDER BY 1 DESC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	reg_date	cohort_size	day_1_retention_...	day_7_retention_...	day_30_retention...
21	2025-08-11	16	6.3	0.0	0.0
22	2025-08-10	16	6.3	0.0	0.0
23	2025-08-09	20	5.0	5.0	0.0
24	2025-08-08	21	19.0	0.0	0.0
25	2025-08-07	15	6.7	0.0	0.0
26	2025-08-06	13	0.0	0.0	0.0

Расчет оттока

Задача: Измерить скорость «утечки» активной базы. Нам нужно понять, какой процент платящих пользователей перестает продлевать подписку или совершать покупки от месяца к месяцу. Высокий Churn Rate — это сигнал о том, что продукт теряет ценность для пользователя быстрее, чем окупаются затраты на его привлечение.

Уровень: Продвинутый
Что тренируем: *приведение дат к периодам (DATE_TRUNC), самосоединение таблиц (Self-Join), расчет разницы между периодами.*

Технический разбор

1. **Ежемесячная активность (CTE MonthlyUsers):** Группируем транзакции по UserID и приводим даты к первому числу месяца, чтобы получить список уникальных плательщиков для каждого периода.
2. **Смещение периодов (Self-Join):** Соединяем таблицу MonthlyUsers (прошлый месяц) с самой собой (текущий месяц) по UserID и условию, что дата второго периода ровно на один месяц больше первого.
3. **Подсчет удержанных (SELECT):** Считаем, сколько пользователей из прошлого месяца успешно «перешли» в следующий. Те, у кого в колонке текущего месяца стоит NULL, — это наш отток.
4. **Финальный расчет:** Вычисляем Churn Rate по формуле (1 - Удержанные / Всего в прошлом месяце) * 100.

Совет

Не путайте «отток пользователей» и «отток выручки» (Revenue Churn). Для полной картины всегда анализируйте обе метрики параллельно. Важно разделять активный отток (пользователь сам отменил подписку) и пассивный (карта просрочена или на ней нет денег). Если пассивный отток велик — это техническая проблема, которую можно решить настройкой каскада ретраев (повторных попыток списания). Анализируйте Churn в связке с причиной отмены, если у вас есть такие данные из опросов.

Расчет оттока

Логика запроса

Мы сопоставляем два списка пользователей: тех, кто был активен в прошлом месяце, и тех, кто остался в текущем. Мы берем «корзину» прошлого месяца за эталон (100%) и ищем этих же людей в «корзине» текущего месяца. Те, кого мы не нашли, пополняют статистику оттока. Разница между этими списками, выраженная в процентах, и есть наш ежемесячный Churn Rate.

Пример кода

```sql
WITH MonthlyUsers AS (
  -- Шаг 1: Собираем уникальных плательщиков по месяцам
  SELECT DISTINCT
    UserID,
    DATE_TRUNC(DATE(DateUTC), MONTH) as month_date
  FROM `project.dataset.payments`
)
SELECT
  m1.month_date as report_month,
  COUNT(DISTINCT m1.UserID) as active_last_month,
  COUNT(DISTINCT m2.UserID) as retained_this_month,
  -- Шаг 2: Рассчитываем процент тех, кто не вернулся в текущем месяце
  ROUND(
    (1 - COUNT(DISTINCT m2.UserID) / CAST(COUNT(DISTINCT m1.UserID) AS FLOAT64)) * 100,
    1
  ) as churn_rate_pct
FROM MonthlyUsers m1
-- Шаг 3: Соединяем месяц с последующим, чтобы проверить наличие юзера
LEFT JOIN MonthlyUsers m2
  ON m1.UserID = m2.UserID
  AND m2.month_date = DATE_ADD(m1.month_date, INTERVAL 1 MONTH)
GROUP BY 1
ORDER BY 1 DESC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON | Execution details

Row	report_month	active_last_month	retained_this_month	churn_rate_pct
1	2025-08-01	928	0	100.0
2	2025-07-01	914	219	76.0
3	2025-06-01	817	217	73.4
4	2025-05-01	852	192	77.5

Накопительный LTV

Задача: Ответить на главный вопрос бизнеса: окупается ли привлечение трафика? Нам нужно рассчитать средний доход от одного пользователя в зависимости от того, сколько месяцев он провел в приложении. Это позволит понять «потолок» стоимости привлечения (CAC) и спрогнозировать выручку от новых когорт.

Уровень: Продвинутый
Что тренируем: *Когортный анализ, разницу дат в месяцах (DATE_DIFF), накопительный итог внутри групп (PARTITION BY).*

Технический разбор

1. **Агрегация UniqueUsers:** Используем MIN(DateUTC) с группировкой по UserID, чтобы гарантировать уникальность пользователя и предотвратить дублирование его платежей при JOIN.
2. **Знаменатель (CohortSizes):** Выделяем месяц из очищенной даты регистрации и считаем количество уникальных ID — это база, на которую мы будем делить всю будущую выручку.
3. **Расчет интервала (DATE_DIFF):** Вычисляем разницу в месяцах между датой платежа и первой датой регистрации, распределяя деньги по «возрасту» пользователя в продукте.
4. **Безопасный JOIN:** Объединяем очищенный список пользователей с таблицей платежей. Теперь каждый платеж гарантированно соответствует только одной строке пользователя.
5. **Оконная функция:** Конструкция SUM(...) OVER последовательно складывает усредненную выручку каждого месяца жизни, формируя финальную кривую накопительного LTV.

Совет

Ищите «точку окупаемости» (Payback Period) — это месяц, в котором кривая LTV пересекает линию стоимости привлечения (CAC). Если за первый месяц (Month 0) пользователь приносит $5, а его привлечение стоит $7 — на первый взгляд это убыток. Но если расчет LTV показывает, что к третьему месяцу он приносит $12, значит, ваш маркетинг эффективен и бизнес масштабируем.

Накопительный LTV

Логика запроса

Сначала мы нормализуем справочник пользователей, сохраняя для каждого ID самую раннюю дату регистрации. Формируем когорты по месяцу регистрации и фиксируем их размер. Для каждой транзакции мы вычисляем, в каком «месяце жизни» пользователя она произошла относительно его первой даты регистрации. Затем мы суммируем выручку для каждого месяца внутри когорты и делим её на количество пользователей в группе. В финале мы используем накопительную сумму для расчета ценности пользователя с течением времени.

Пример кода

```sql
WITH UniqueUsers AS (
  -- Шаг 0: Очищаем справочник юзеров. Берем самую раннюю дату регистрации.
  SELECT
    UserID,
    MIN(DateUTC) as FirstDateUTC
  FROM `project.dataset.users`
  GROUP BY UserID
),
CohortSizes AS (
  -- Шаг 1: Определяем размер каждой когорты (знаменатель)
  SELECT
    DATE_TRUNC(DATE(FirstDateUTC), MONTH) as reg_month,
    COUNT(DISTINCT UserID) as users_in_cohort
  FROM UniqueUsers
  GROUP BY reg_month
),
RevenueSteps AS (
  -- Шаг 2: Распределяем выручку по месяцам жизни.
  SELECT
    DATE_TRUNC(DATE(u.FirstDateUTC), MONTH) as reg_month,
    DATE_DIFF(DATE(p.DateUTC), DATE(u.FirstDateUTC), MONTH) as lifetime_month,
    SUM(p.RevenueUSD) as monthly_revenue
  FROM UniqueUsers u
  JOIN `project.dataset.payments` p
    ON u.UserID = p.UserID
  GROUP BY reg_month, lifetime_month
)
SELECT
  r.reg_month,
  r.lifetime_month,
  -- Шаг 3: Считаем накопительный LTV
  ROUND(
    SUM(r.monthly_revenue / c.users_in_cohort) OVER (
      PARTITION BY r.reg_month
      ORDER BY r.lifetime_month
    ), 2
  ) as cumulative_ltv
FROM RevenueSteps r
JOIN CohortSizes c ON r.reg_month = c.reg_month
ORDER BY reg_month DESC, lifetime_month ASC;
```

Ожидаемый результат

Job information | Results | Visualization | JSON

Row	reg_month	lifetime_month	cumulative_ltv
11	2025-04-01	0	2.68
12	2025-04-01	1	4.36

Послесловие

Вы закрыли последнюю страницу, но настоящая работа начинается только сейчас.

Мы прошли путь от проверки дублей до расчета LTV. Если вы честно прогнали все скрипты в BigQuery — вы неплохо потренировались для решения практических, близких к рабочим задач.

И хотя в этом сборнике данные были “послушными”, важно помнить, что реальности все будет не так радужно — у вас чаще всего не будет таких красивых и удобных таблиц, логи будут пропадать, таймзоны — смешиваться, а платежные системы — присылать мусор и ошибки.

Но секрет в том, что SQL — это всего лишь способ доставки ваших мыслей до машины. А этот учебник пытался вас научить самому главному — мыслить как аналитик. Самое важное происходило у вас голове в тот момент, когда вы решали, какую группу пользователей отсечь и как сгладить тренд. Задачи могут отличаться, данные скорее всего будут отличаться, рабочее окружение вероятно будет другим - но способ мышления и подхода к решению задачи останется тем же.

Мышление аналитика — это не про знание всех функций SQL, а скорее про привычку критически мыслить и не принимать ничего на веру. Теперь вы вооружены и готовы к встрече с реальным продакшеном. Не ждите идеальных условий, создавайте их сами через чистый код и логические проверки. Ошибайтесь, перепроверяйте себя и помните: самый ценный результат — не красивый график, а правильное решение, которое за ним стоит.

Помните то чувство, когда за миллионами строк логов вдруг вырисовывается четкая линия поведения пользователей или точка окупаемости проекта? Это и есть магия профессии аналитика.

Kanstantsin Hupalau